MedTrack: A Cloud-Based Healthcare Management System Using AWS Using AWS

Project Description:

MedTrack is a cloud-based healthcare management system designed to improve communication and coordination between doctors and patients through a centralized digital platform. The system allows patients to register, log in, book medical appointments, and submit diagnosis reports online, while doctors can access patient records and manage appointments efficiently.

The application is developed using the Flask backend framework and deployed on Amazon EC2 for scalable cloud hosting. DynamoDB is used as the primary database to store user data, appointment details, and medical records. The platform enables real-time data retrieval and updates, ensuring that doctors and patients always have access to the latest information.

MedTrack enhances healthcare accessibility by providing a simple and efficient interface for appointment booking and medical record management. Security is maintained through AWS Identity and Access Management (IAM), ensuring that only authorized users can access sensitive patient data. By leveraging cloud technology, MedTrack delivers a reliable, scalable, and secure healthcare management solution.

Scenario 1: Efficient Appointment Booking System for Patients

In the MedTrack system, AWS EC2 provides a reliable infrastructure to manage multiple patients accessing the platform simultaneously. For example, a patient can log in, navigate to the appointment booking page, and easily submit a request for an appointment. Flask handles backend operations, efficiently retrieving and processing user data in real-time. The cloud-based architecture allows the platform to handle a high volume of appointment requests during peak periods, ensuring smooth operation without delays.

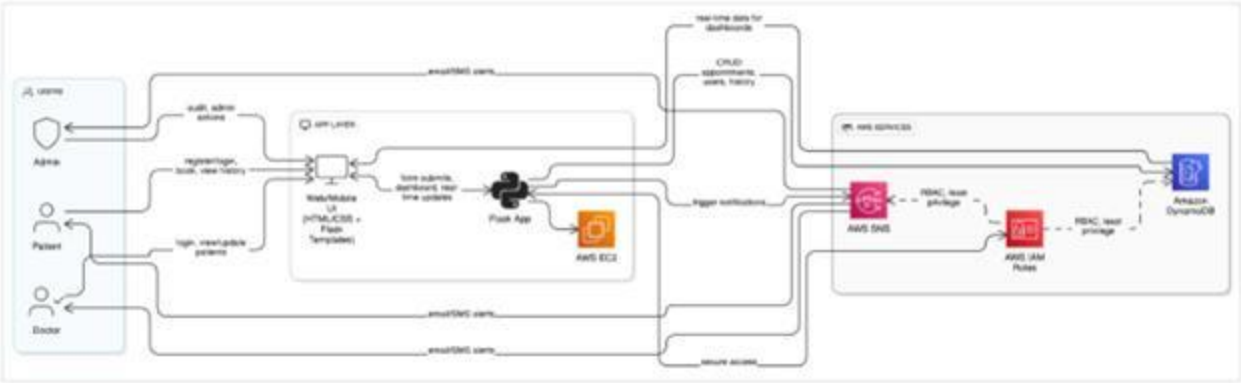
Scenario 2: Secure User Management with IAM

MedTrack utilizes AWS IAM to manage user permissions and ensure secure access to the system. For instance, when a new patient registers, an IAM user is created with specific roles and permissions to access only the features relevant to them. Doctors have their own IAM configurations, allowing them access to patient records and appointment details while maintaining strict security protocols. This setup ensures that sensitive data is accessible only to authorized users.

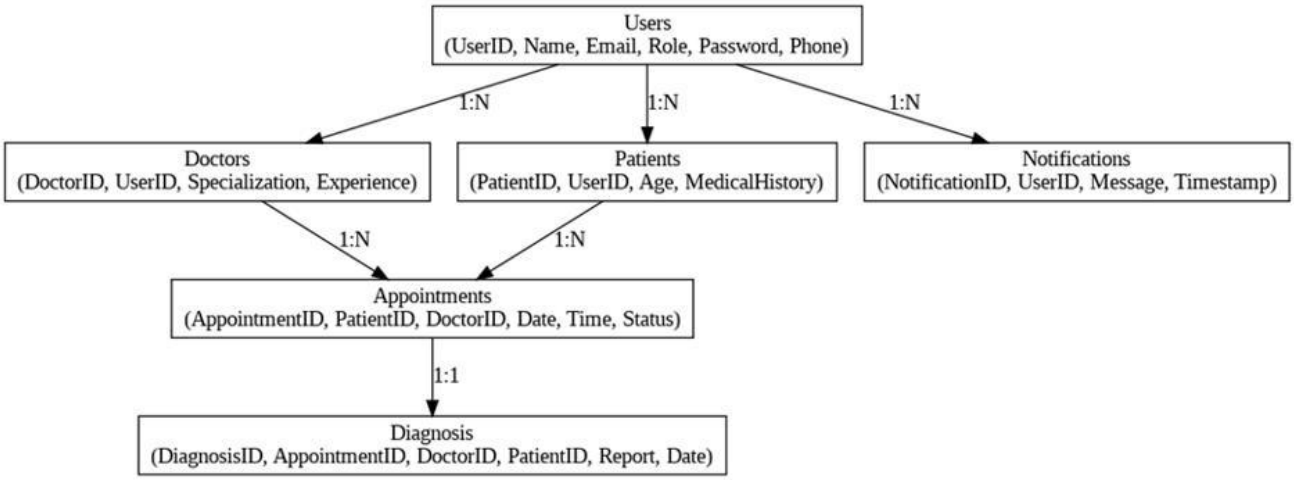
Scenario 3: Easy Access to Medical History and Resources

The MedTrack system provides doctors and patients with easy access to medical histories and relevant resources. For example, a doctor logs in to view a patient's medical history and upcoming appointments. They can quickly access, and update records as needed. Flask manages real-time data fetching from DynamoDB, while EC2 hosting ensures the platform performs seamlessly even when multiple users access it simultaneously, offering a smooth and uninterrupted user experience.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

1. **AWS Account Setup**: AWS Account Setup
2. **Understanding IAM**: IAM Overview
3. **Amazon EC2 Basics**: EC2 Tutorial
4. **DynamoDB Basics**: DynamoDB Introduction
5. **SNS Overview**: SNS Documentation
6. **Git Version Control**: Git Documentation

Project WorkFlow:

1. AWS Account Setup and Login

Activity 1.1: Set up an AWS account if not already done**.**

Activity 1.2: Log in to the AWS Management Console

2. DynamoDB Database Creation and Setup

Activity 2.1: Create DynamoDB tables for storing application data.

Activity 2.2: Configure attributes for **Patients, Doctors, Appointments, and MedicalRecords tables**.

3. SNS Notification Setup

Activity 3.1: Create an SNS Topic named **AppointmentNotification** to send notifications to users.

Activity 3.2: Subscribe patient or doctor email addresses to receive appointment confirmations, updates, or cancellations**.**

4. Backend Development and Application Setup

Activity 4.1:Develop the Backend Using Flask.

Activity 4.2: Integrate AWS services such as **DynamoDB and SNS using boto3**..

5. IAM Role Setup

Activity 5.1: Create IAM Role

Activity 5.2: Attach Policies

6. EC2 Instance Setup

Activity 6.1: Launch an EC2 instance to host the Flask application.

Activity 6.2: Configure security groups for HTTP, and SSH access.

7. Deployment on EC2

Activity 7.1: Upload Flask application files to the EC2 instance

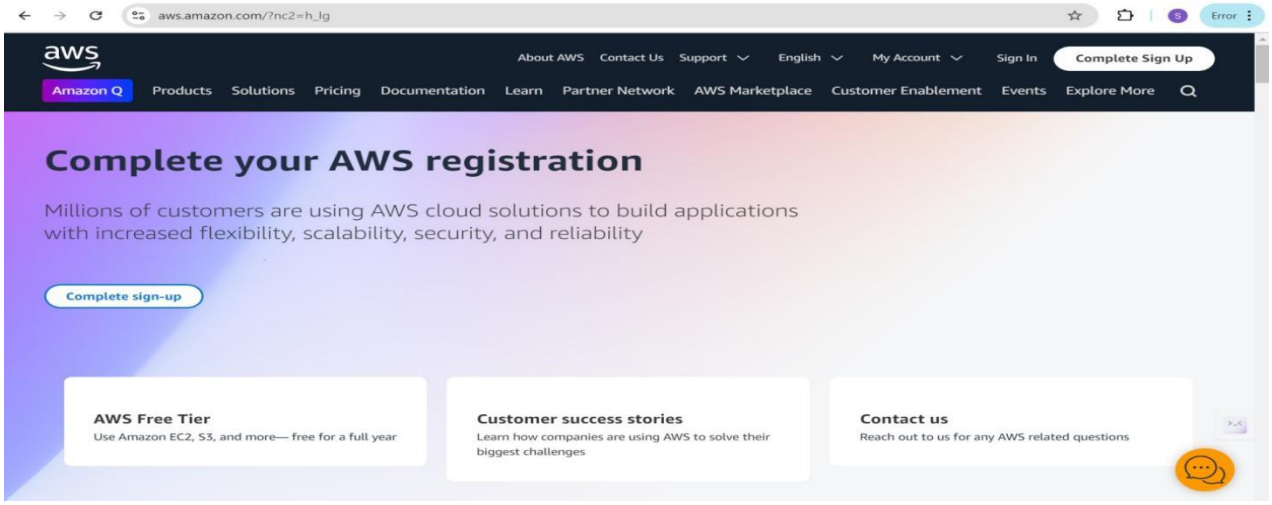
Activity 7.2: Run the Flask App

8. Testing and Deployment

Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.

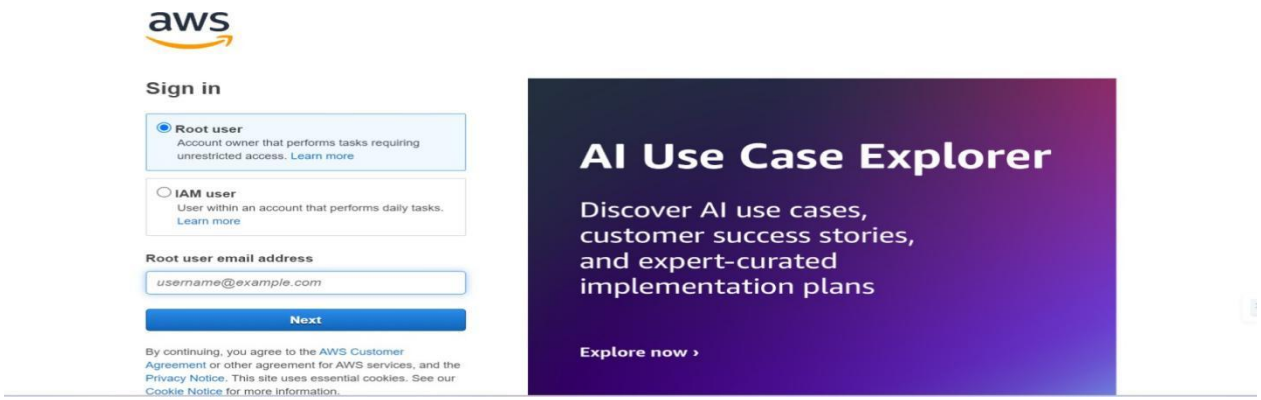
Milestone 1: AWS Account Setup and Login

- **Activity 1.1: Set up an AWS account if not already done.**
  - Sign up for an AWS account and configure billing settings.



- **Activity 1.2: Log in to the AWS Management Console**

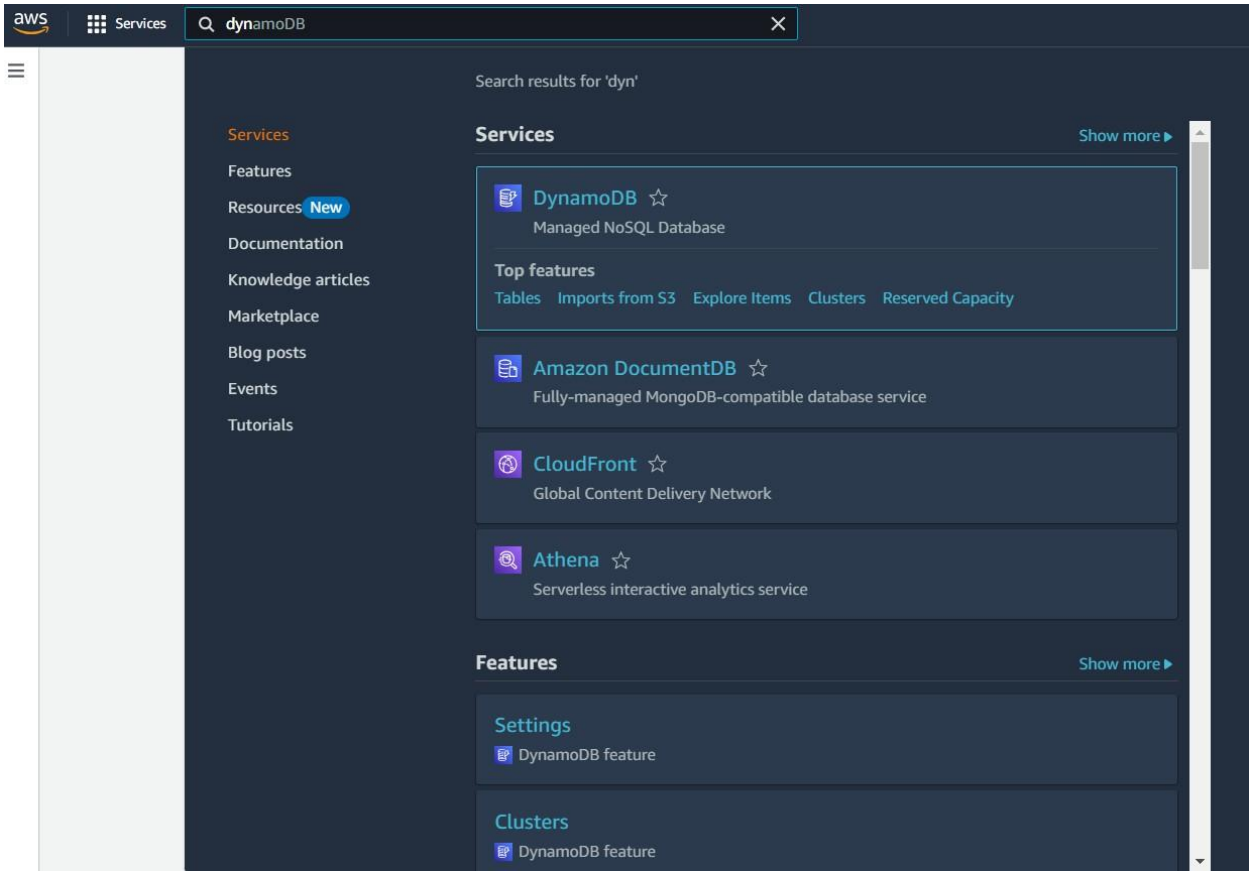
  - After setting up your account, log in to the AWS Management Console.

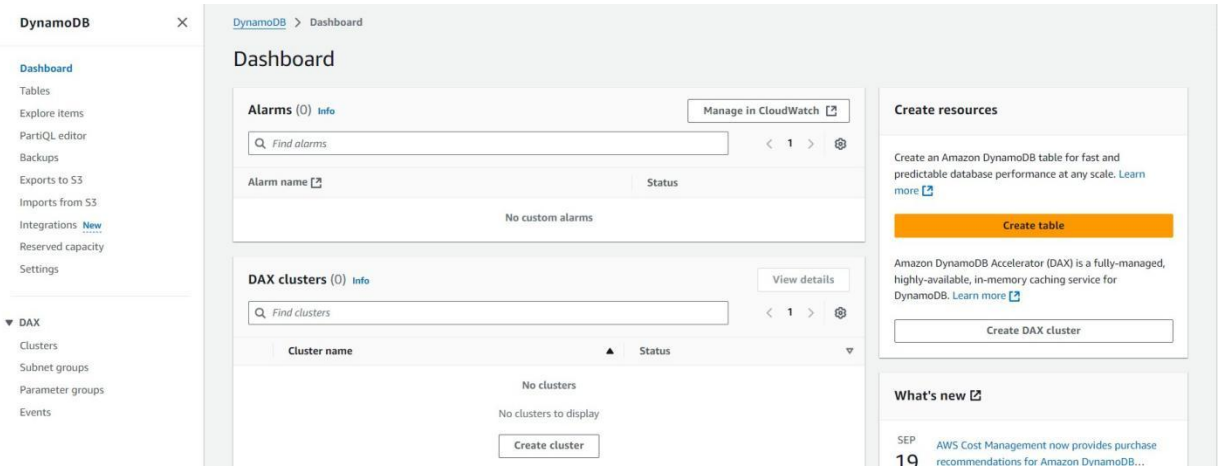


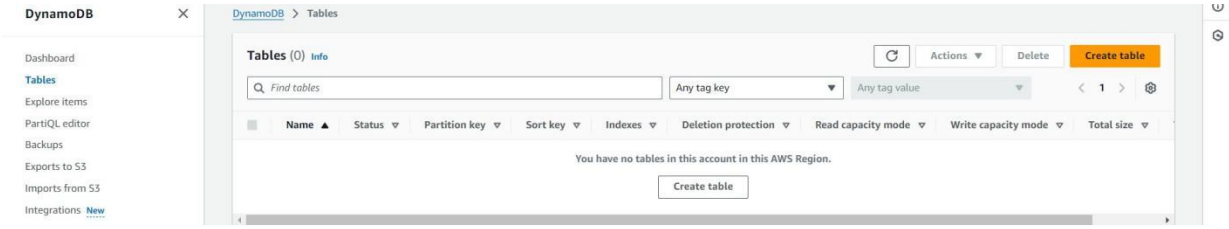
Milestone 2: DynamoDB Database Creation and Setup

- **Activity 2.1:Navigate to the DynamoDB**

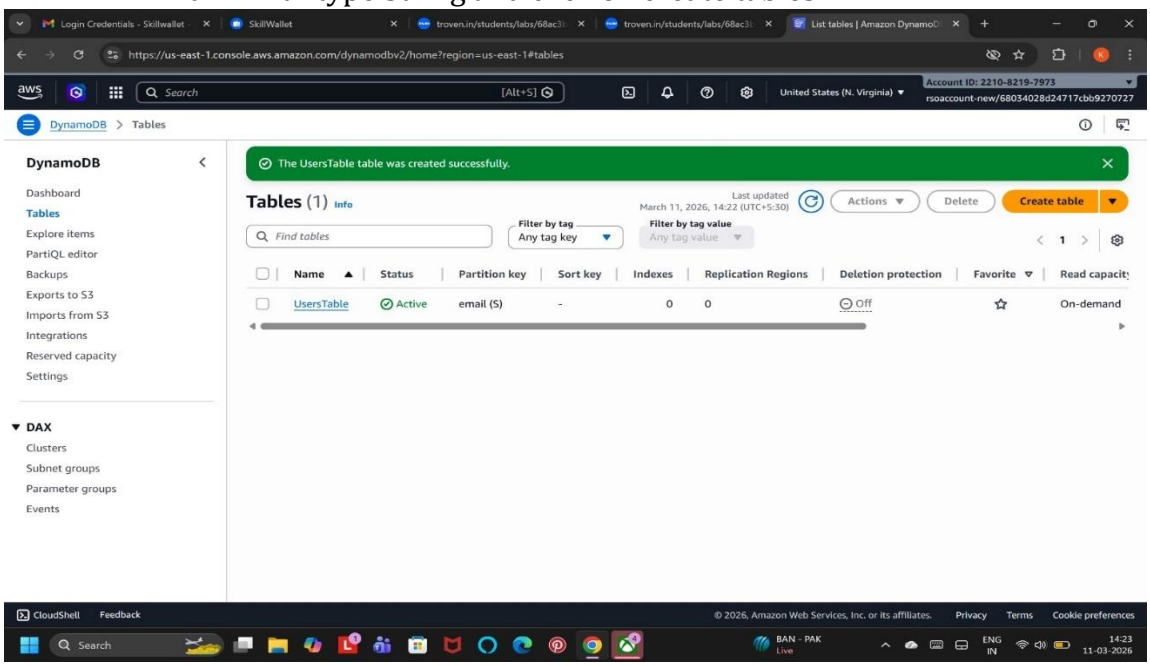
- In the AWS Console, navigate to DynamoDB and click on create tables.







- **Activity 2.2: Create DynamoDB tables for storing registration details and booking records.**
    - Create a Users table for storing registered user details with partition key "Email" with type String and click on create tables.



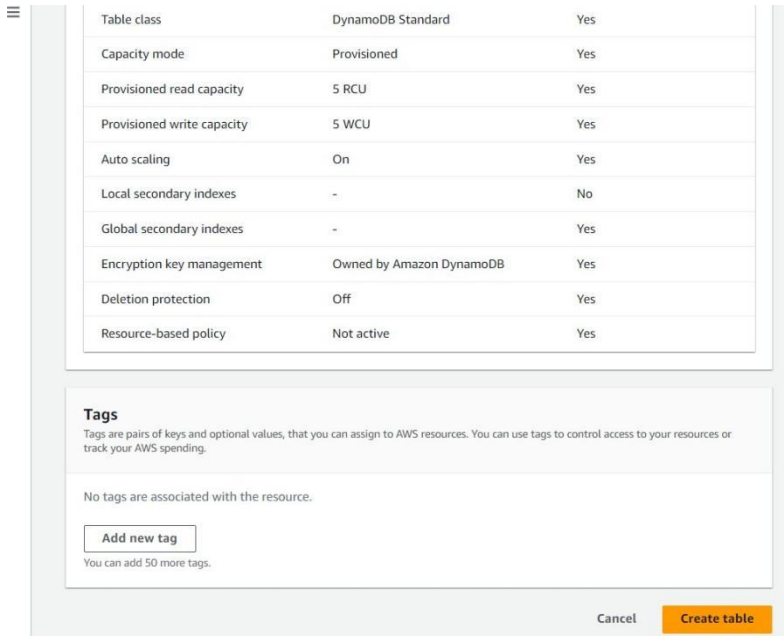
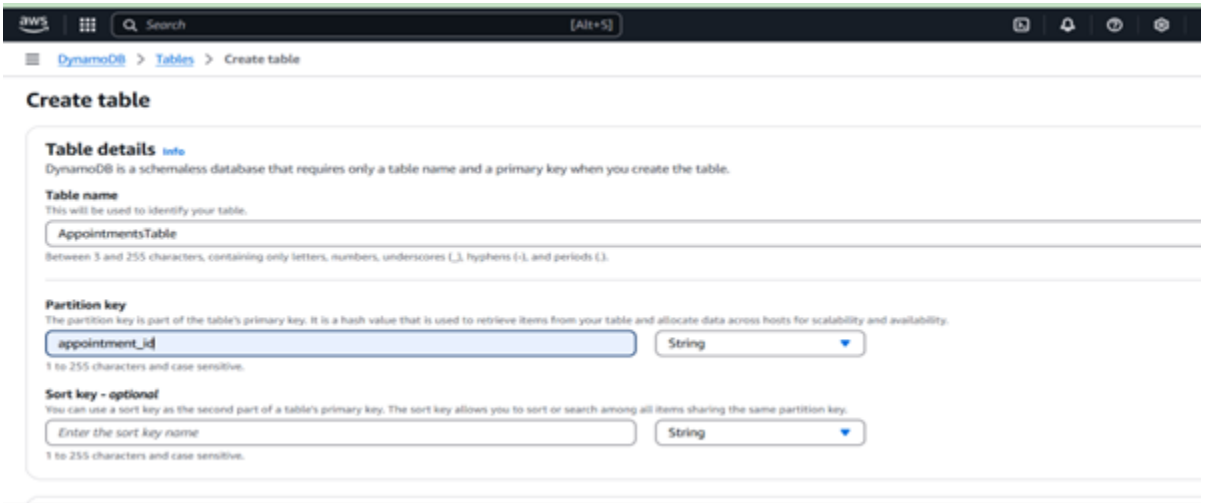


Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

- Create AppointmentsTable with partition key “appointment_id” with type String and click on create tables..



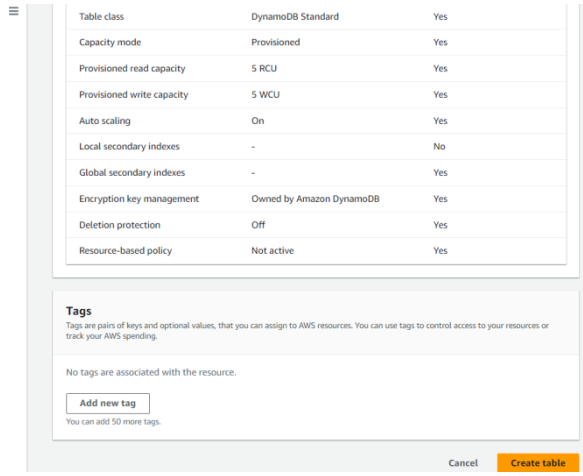
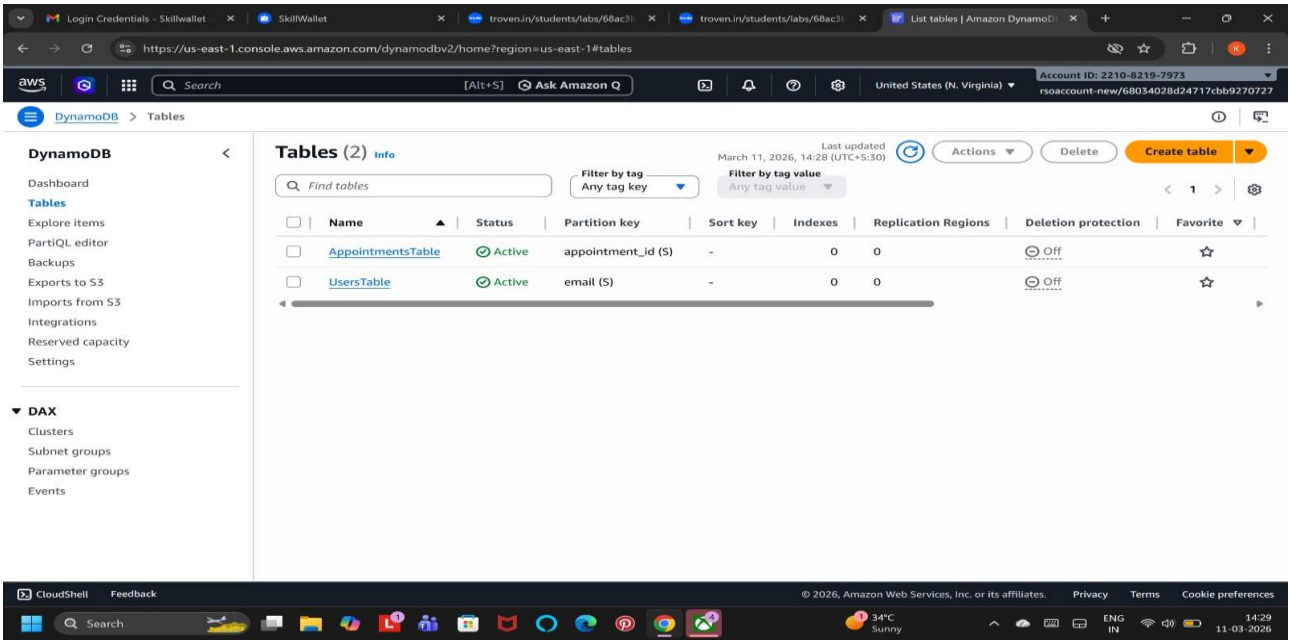
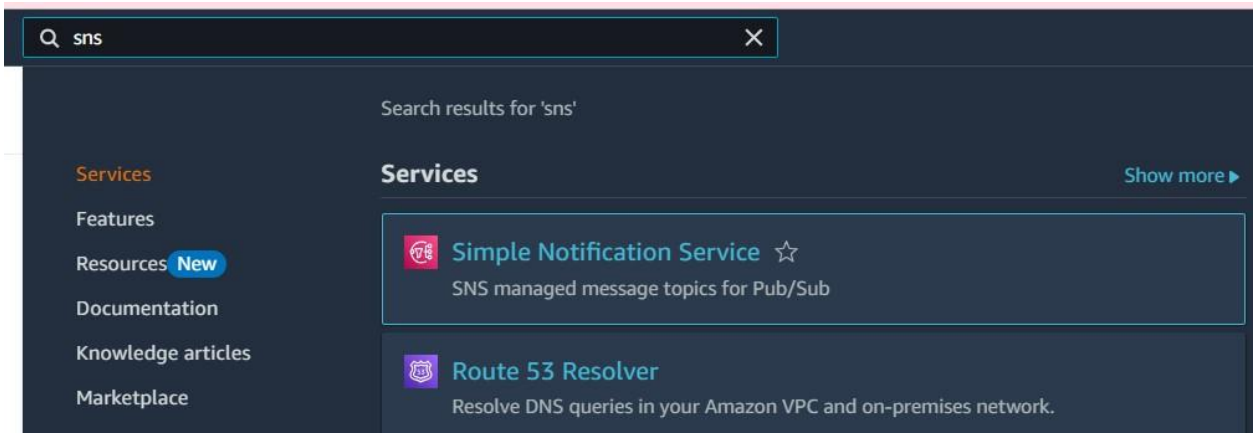


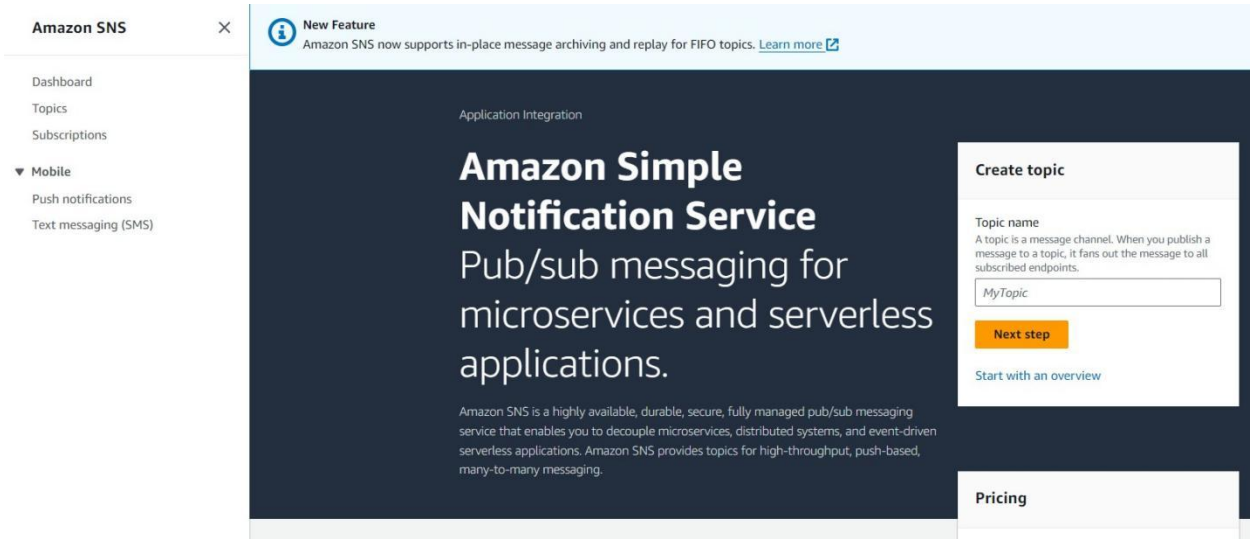
Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes



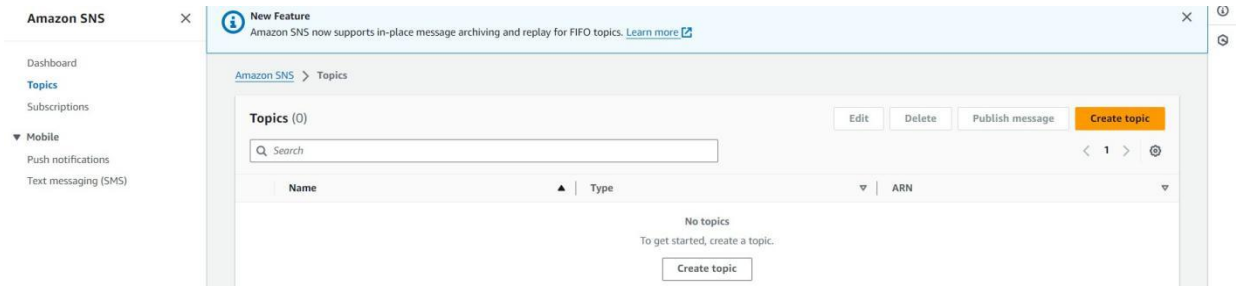
Milestone 3: SNS Notification Setup

- **Activity 3.1: Create an SNS Topic named BookingConfirmation for sending notifications.**
    - In the AWS Console, search for SNS and navigate to the SNS Dashboard.

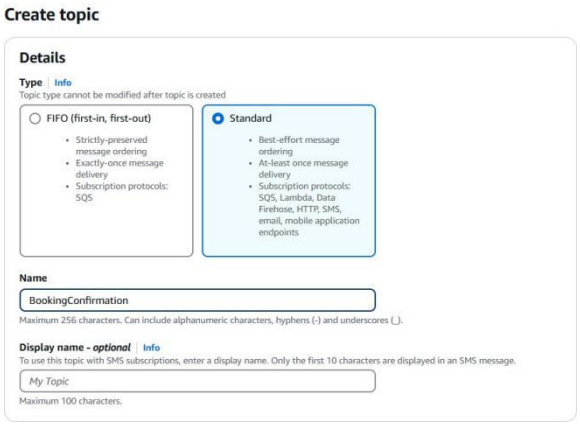


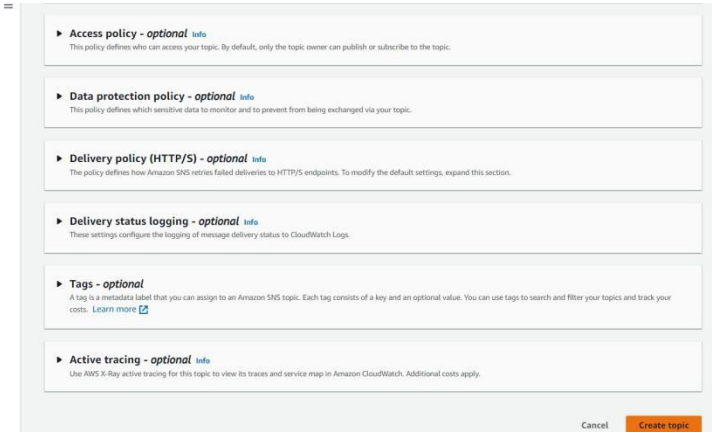


- Click on **Create Topic** and choose a name for the topic.

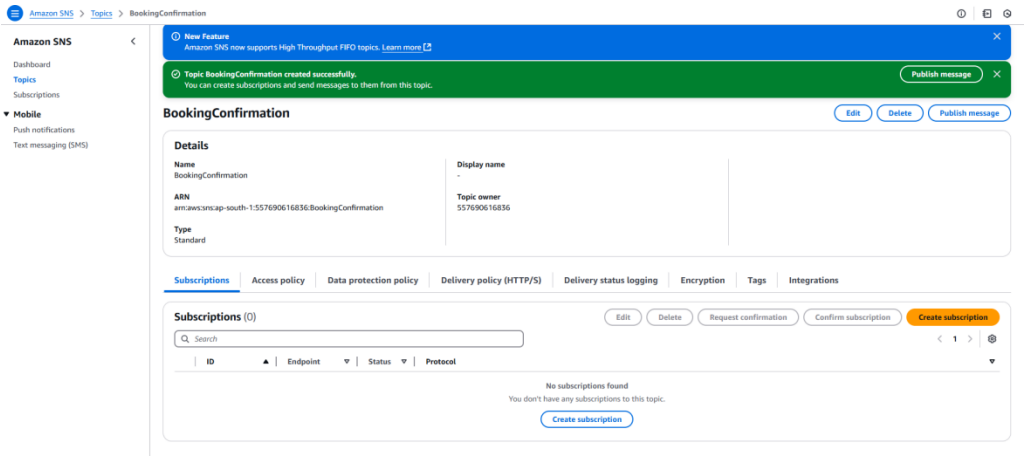


- Choose Standard type for general notification use cases and Click on Create Topic.

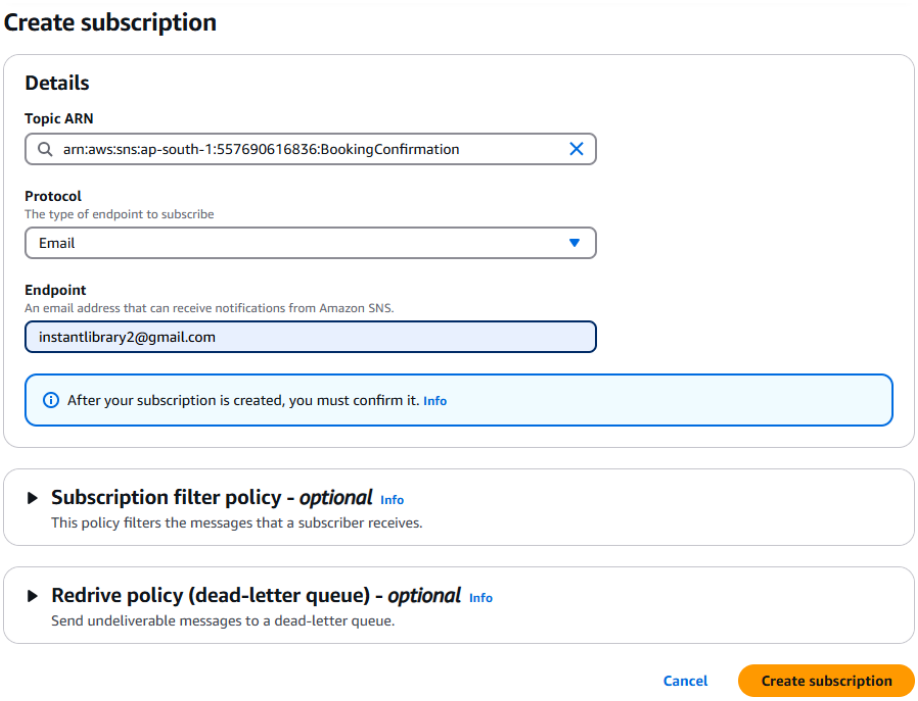




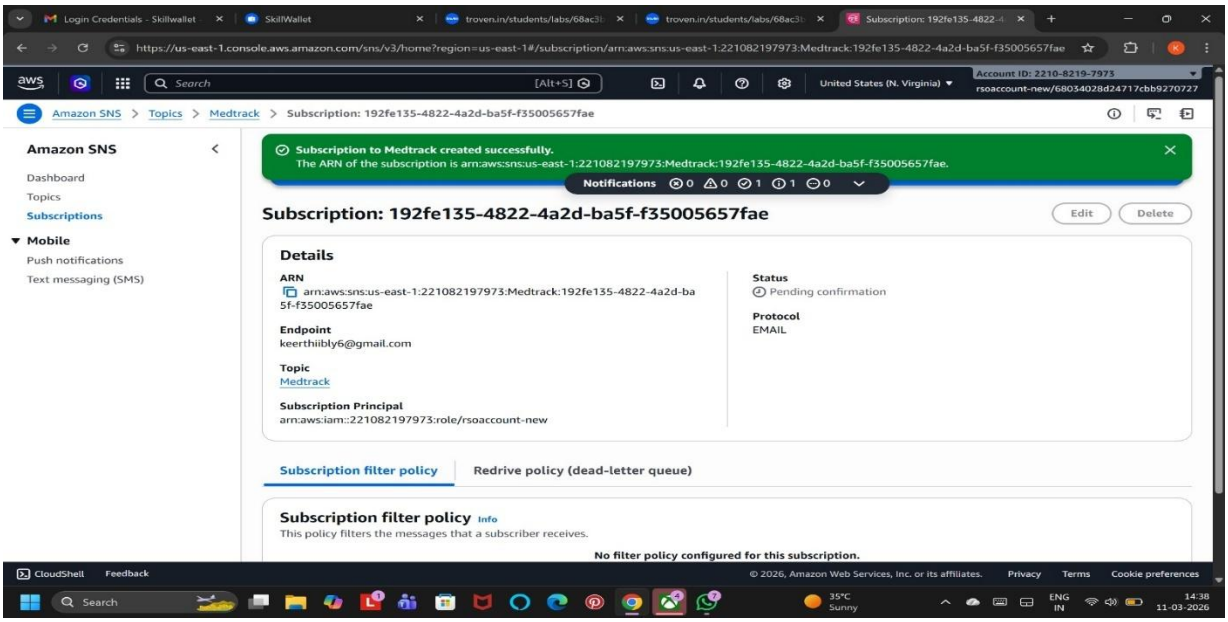
- Configure the SNS topic and note down the **Topic ARN**.



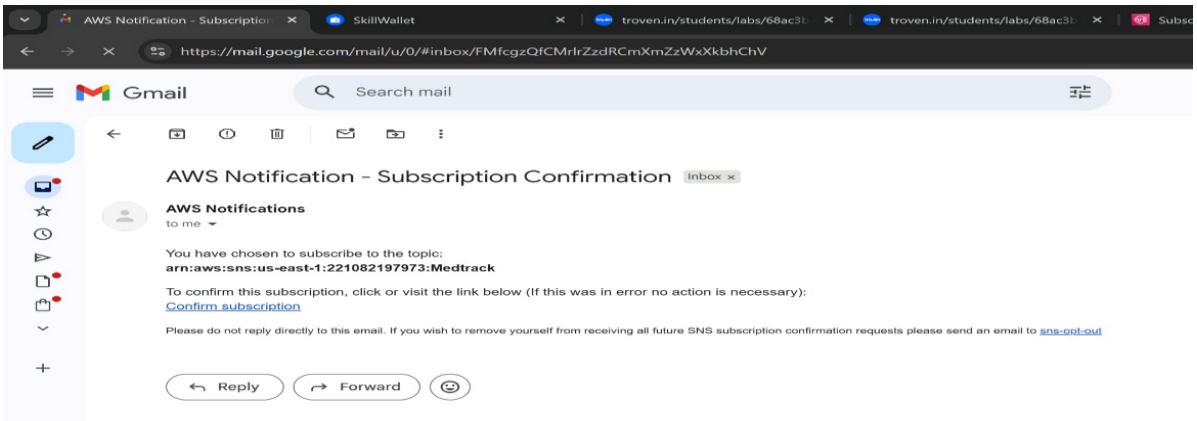
- **Activity 3.2: Subscribe email to the topic to receive booking and cancellation alerts.**
    - Subscribe users (or admin staff) to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails**.**

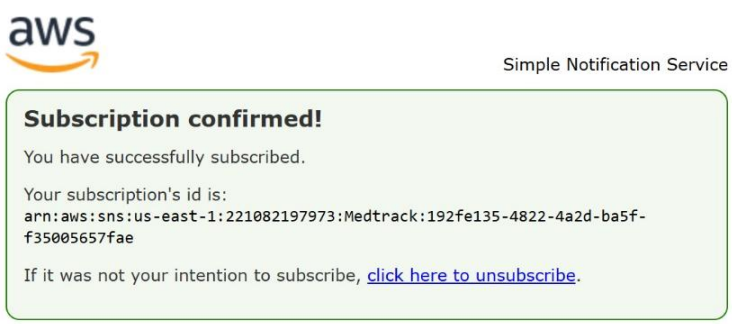


- After subscription request for the mail confirmation

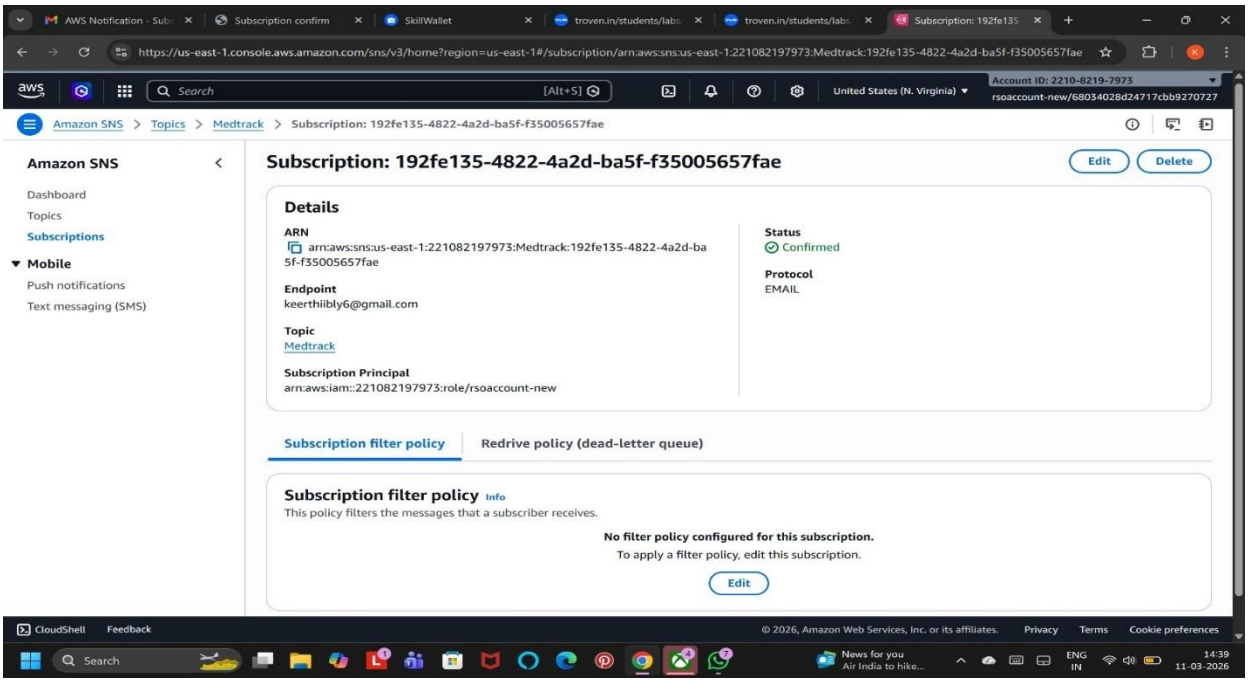


- Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.



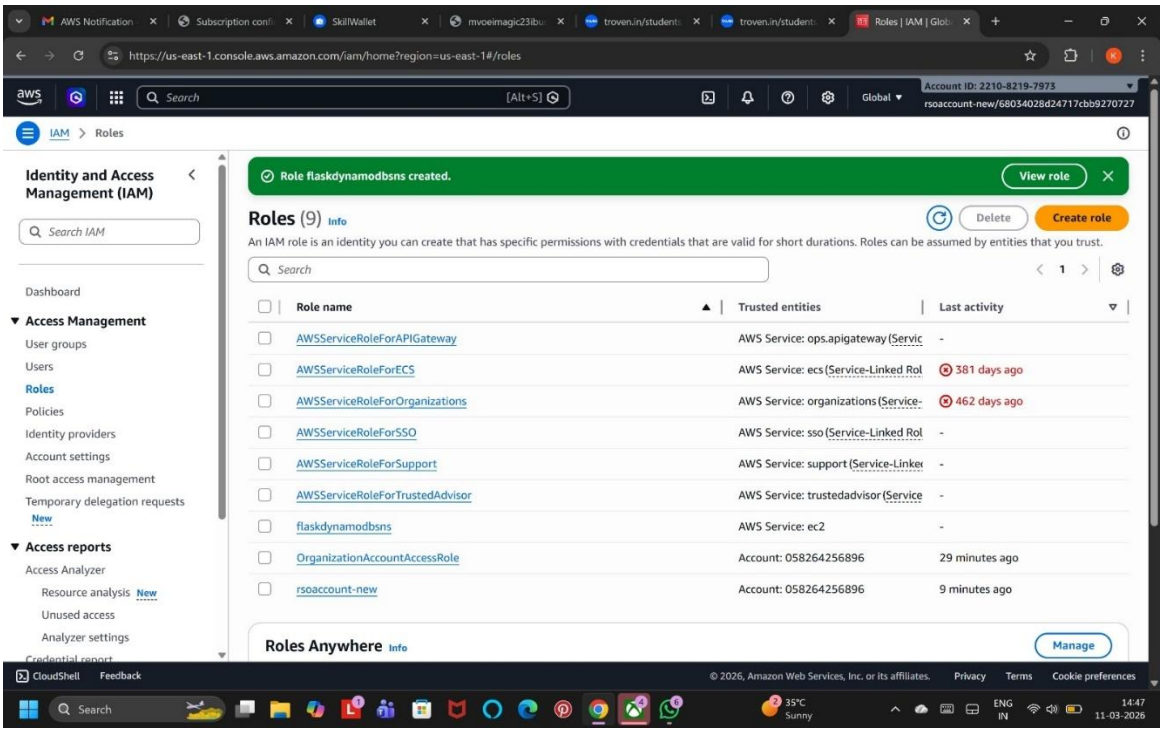


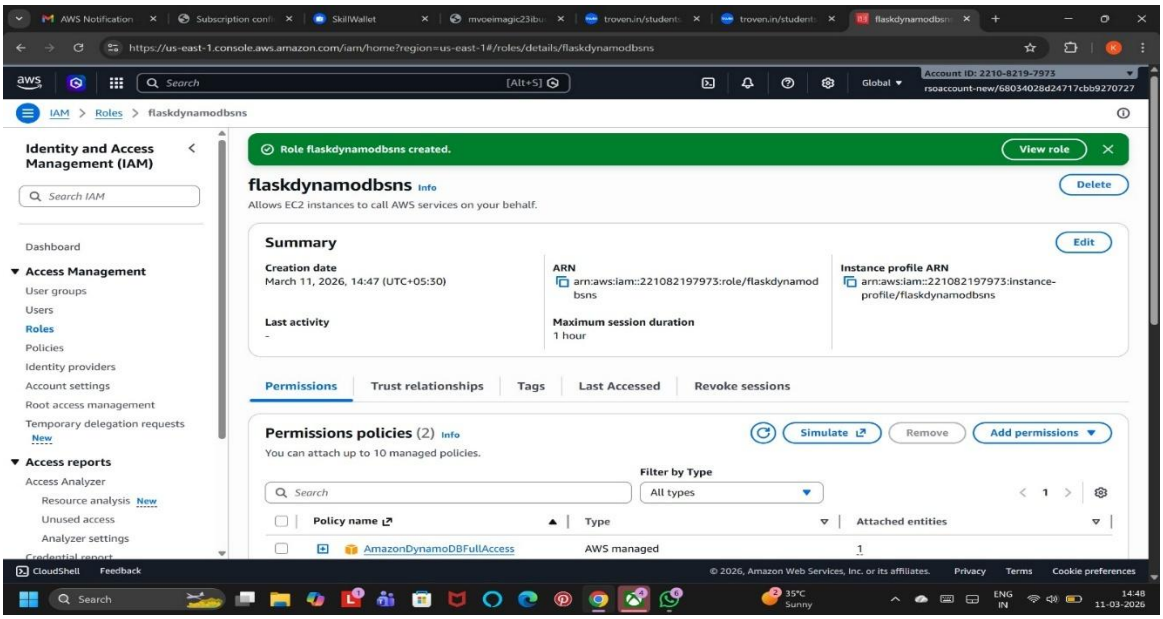
- Successfully done with the SNS mail subscription and setup, now store the ARN link.



Milestone 4: Create IAM Role

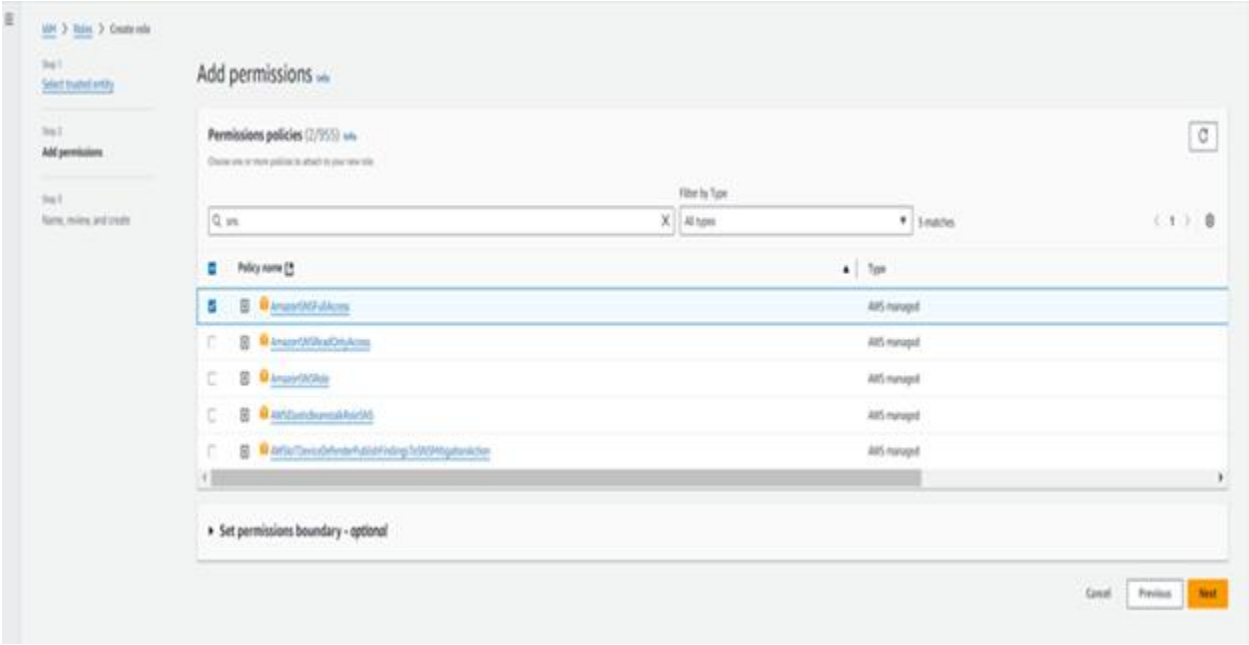
Activity 4.1: In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS



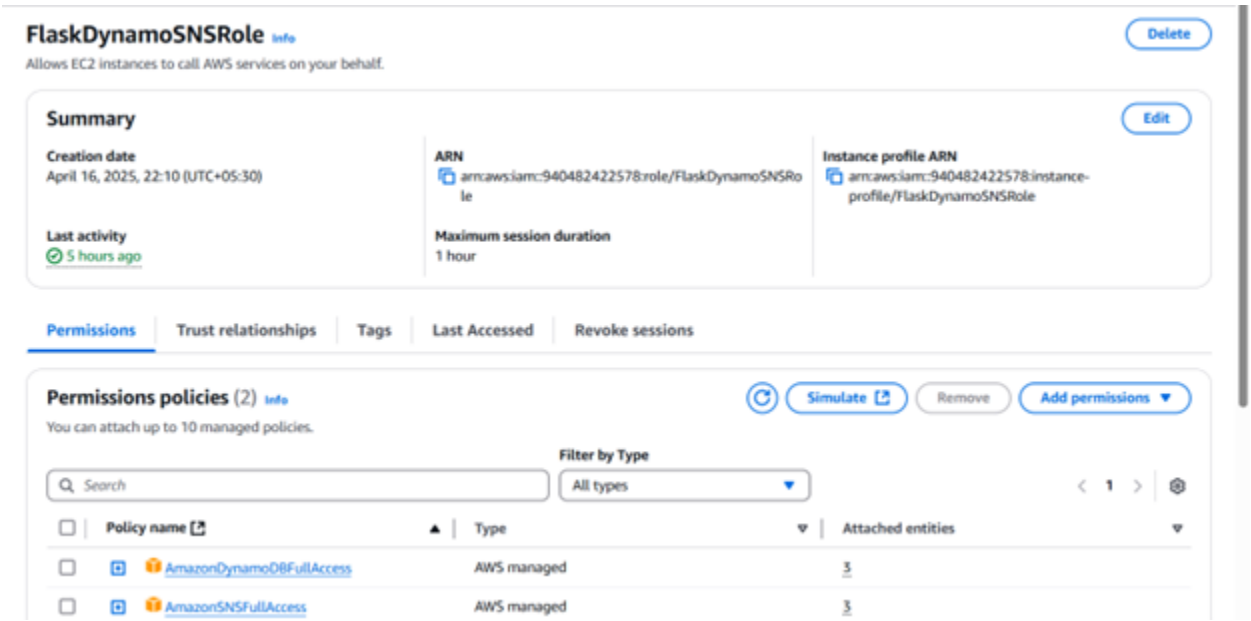


Activity 4.2: Attach Policies

- AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.
- AmazonSNSFullAccess: Grants EC2 the ability to send notifications via SNS.



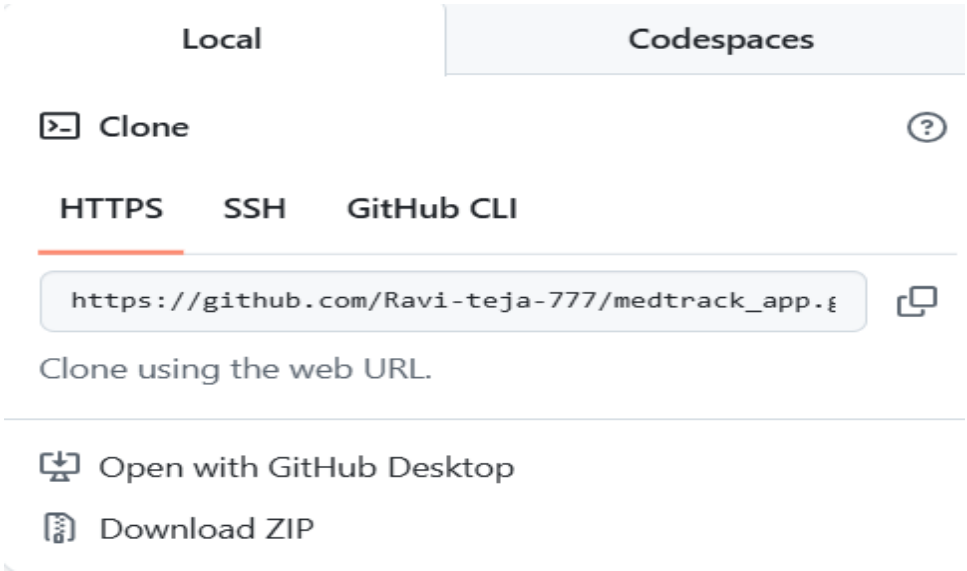
- To create a role named **flaskdynamodbsns,** go to the AWS IAM console, create a new role, and assign DynamoDBFullAccess and SNSFullAccess policies. Name the role flaskdynamodbsns and attach it to the necessary AWS services. This role will allow your Flask app to interact with both DynamoDB and SNS seamlessly.



Milestone 5: EC2 Instance setup

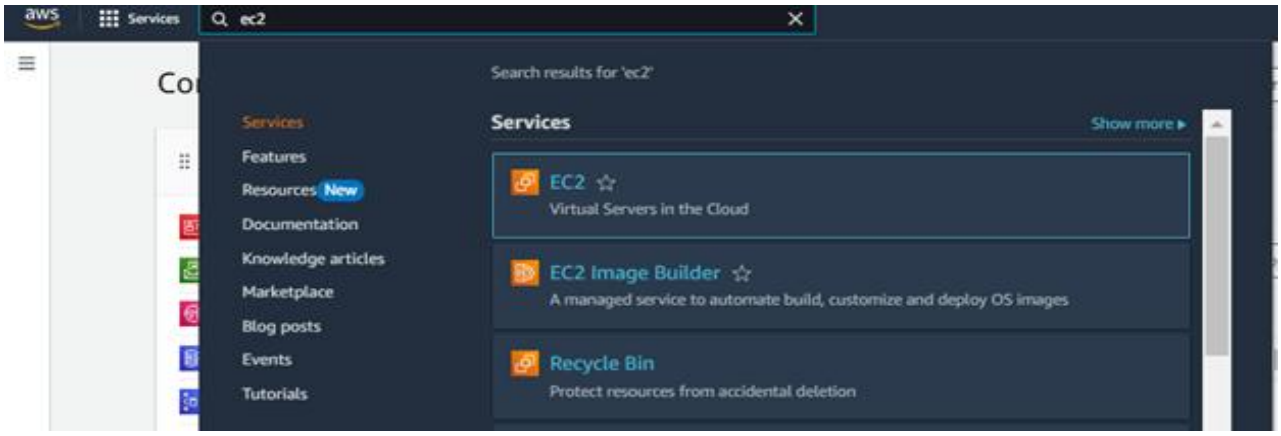
Activity 5.1: Load your Project Files to GitHub

- To upload your Flask app and HTML files to GitHub, initialize the repository using git init, commit the files with git add . && git commit -m "Initial commit", and push them using git push -u origin main. This will load your project onto GitHub.



Activity 5.2: Launch an EC2 instance to host the Flask

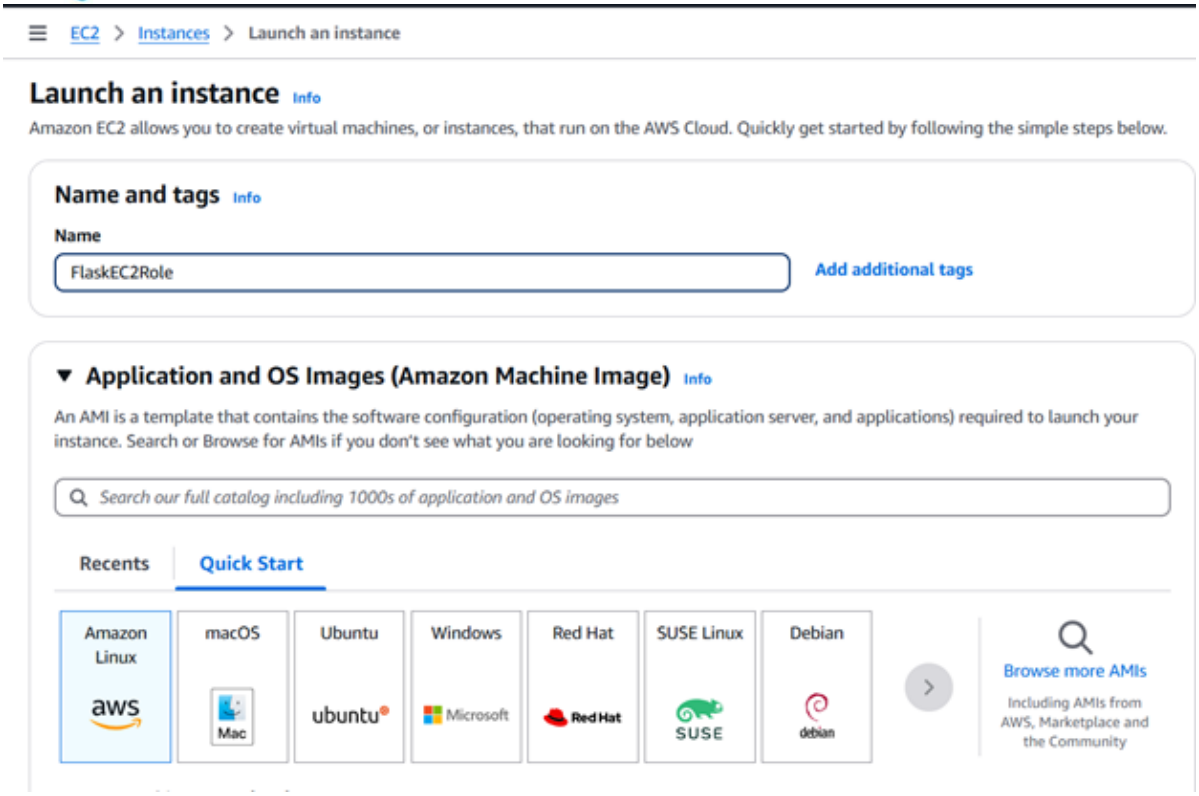
- In the AWS Console, navigate to EC2 and launch a new instance.



- To launch an EC2 instance with the name **flaskec2role**, follow these steps:

  1. Go to the **AWS EC2 Dashboard** and click on **Launch Instance**.

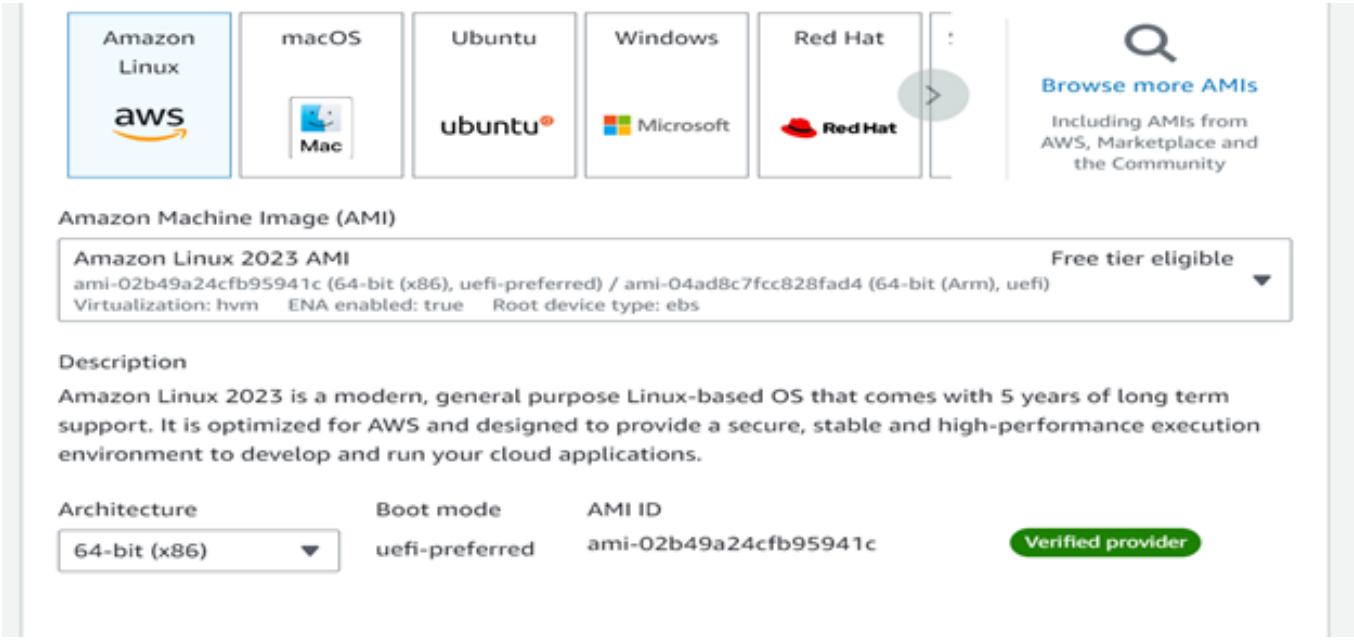
  2. Select your desired AMI, instance type, configure instance details, and under **IAM role**, choose the role **flaskec2role**. Finally, launch the instance.



- To launch an EC2 instance with **Amazon Linux 2** or **Ubuntu** as the AMI and **t2.micro** as the instance type (free-tier eligible):

  1. In the **Launch Instance** wizard, choose **Amazon Linux 2** or **Ubuntu** from the available AMIs.

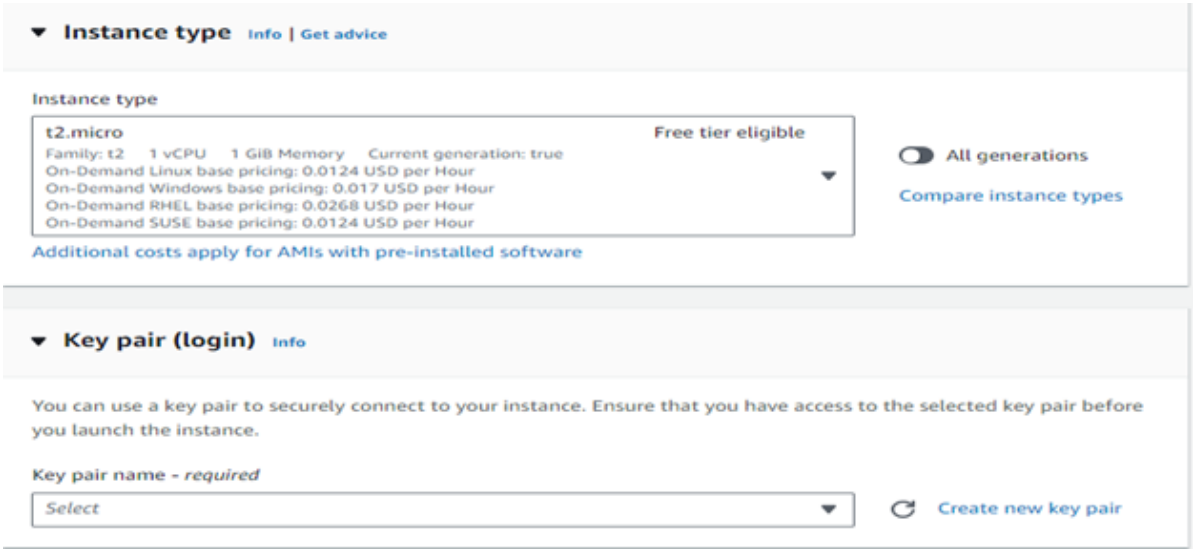
  2. Select **t2.micro** as the instance type, which is free-tier eligible, and continue with the configuration and launch steps.

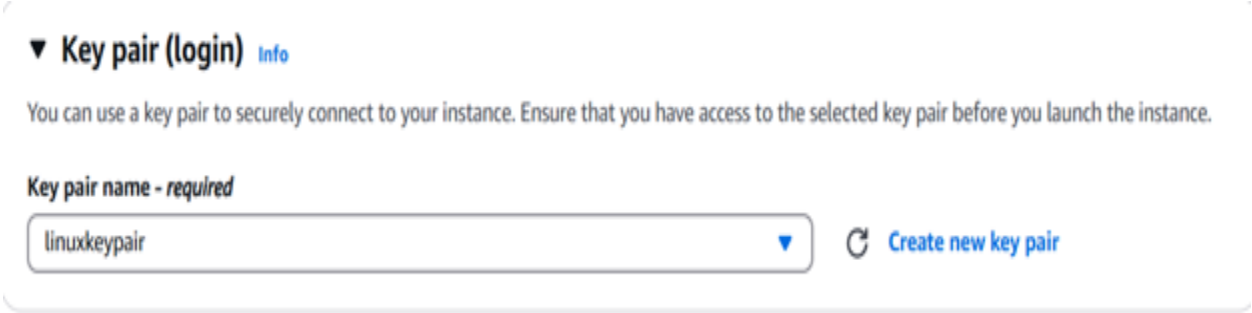


- To create and download the key pair for server access:

  1. In the **Launch Instance** wizard, under the **Key Pair** section, click **Create a new key pair**.

  2. Name your key pair (e.g., **flaskkeypair**) and click **Download Key Pair**. This will download the .pem file to your system, which you will use to access the EC2 instance securely via SSH.





Activity 5.3: Configure security groups for HTTP, and SSH access.

- **For network settings during EC2 instance launch:**

  1. **In the** Network Settings **section, select the** VPC **and** Subnet **you wish to use (if unsure, the default VPC and subnet should work).**

  2. **Ensure** Auto-assign Public IP **is enabled so your instance can be accessed from the internet.**

  3. **In** Security Group**, either select an existing one or create a new one that allows SSH (port 22) access to your EC2 instance for remote login.**

Network settings Info

VPC - *required* | Info

vpc-03cdc7b6f19dd7211 (default)
172.31.0.0/16

Subnet | Info

No preference — Create new subnet

Auto-assign public IP | Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups) | Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group | Select existing security group

Security group name - *required*

launch-wizard

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and ._-:/()#,@[]+=&;{}!$*

Description - *required* | Info

launch-wizard created 2024-10-13T17:49:56.622Z

Inbound Security Group Rules

Security group rule 1 (TCP, 22, 0.0.0.0/0) — Remove

Type	Protocol	Port range
ssh	TCP	22

Source type	Source	Description - *optional*
Anywhere	Add CIDR, prefix list or security 0.0.0.0/0	e.g. SSH for admin desktop

Security group rule 2 (TCP, 80, 0.0.0.0/0) — Remove

Type	Protocol	Port range
HTTP	TCP	80

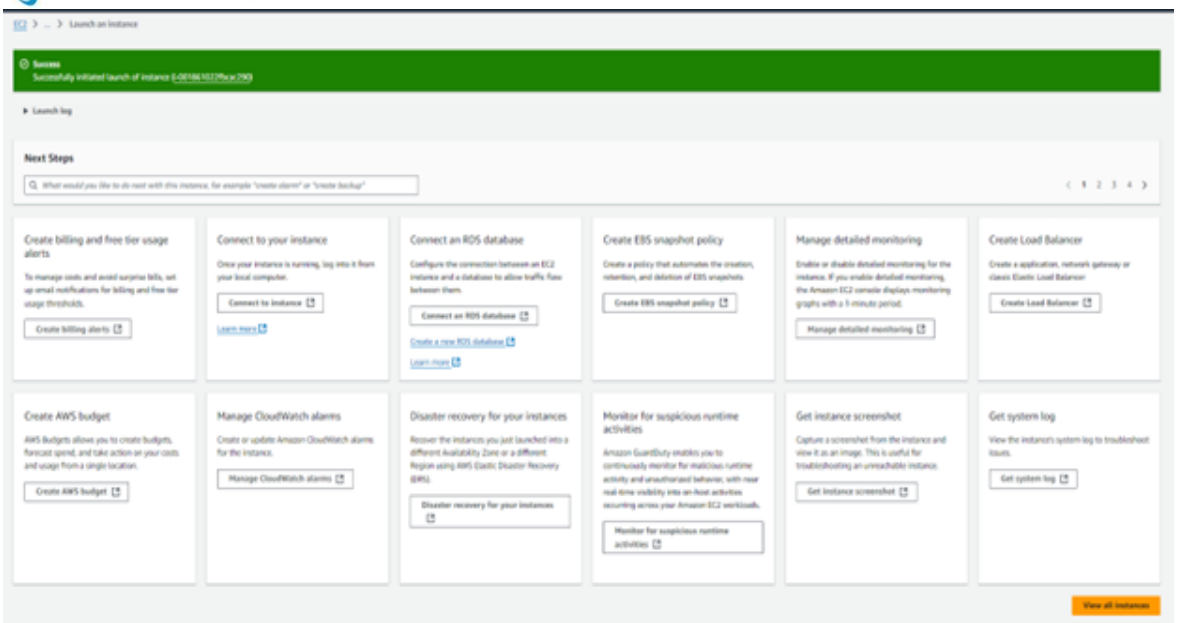
Source type	Source	Description - *optional*
Custom	Add CIDR, prefix list or security 0.0.0.0/0	e.g. SSH for admin desktop

Security group rule 3 (TCP, 5000, 0.0.0.0/0) — Remove

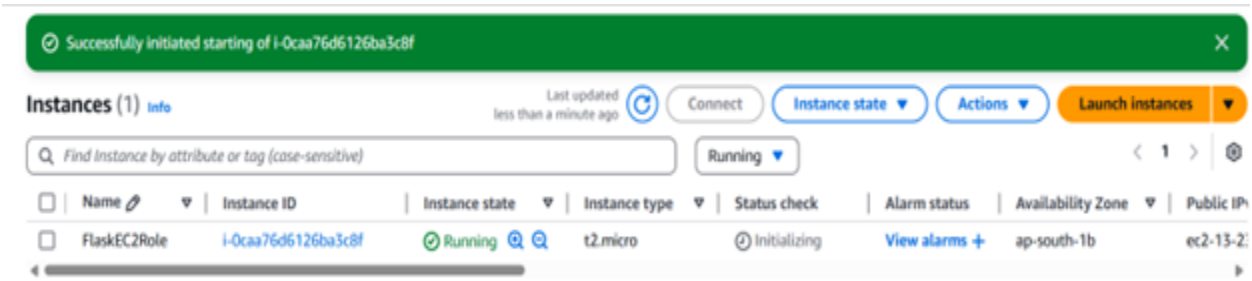
Type	Protocol	Port range
Custom TCP	TCP	5000

Source type	Source	Description - *optional*
Custom	Add CIDR, prefix list or security 0.0.0.0/0	e.g. SSH for admin desktop

Add security group rule

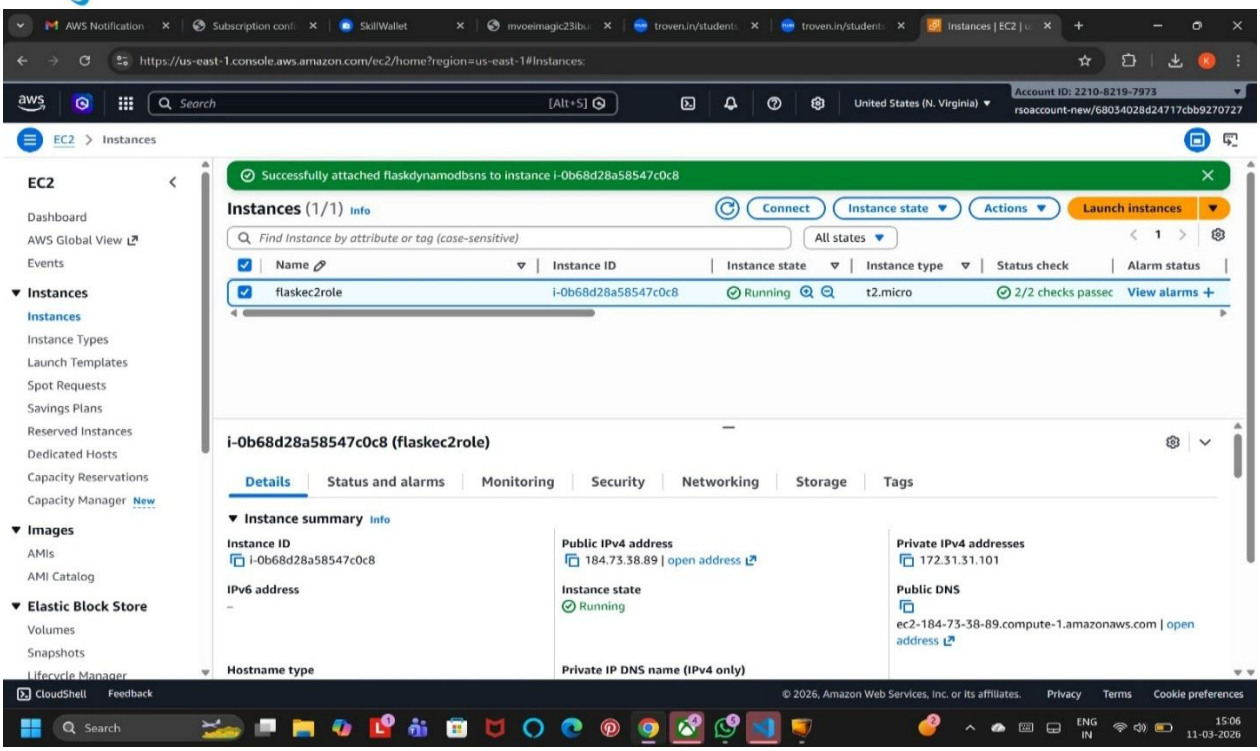


To connect to EC2 using EC2 Instance Connect, start by ensuring that an IAM role is attached to your EC2 instance. You can do this by selecting your instance, clicking on Actions, then navigating to Security and selecting Modify IAM Role to attach the appropriate role. After the IAM role is connected, navigate to the EC2 section in the AWS Management Console. Select the EC2 instance you wish to connect to. At the top of the EC2 Dashboard, click the Connect button. From the connection methods presented, choose EC2 Instance Connect. Finally, click Connect again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.



The EC2 instance you are launching is configured with Amazon Linux 2 or Ubuntu as the AMI, t2.micro as the instance type (free-tier eligible), and flaskec2role IAM role for appropriate permissions.

The flaskkeypair key pair is created for secure server access via SSH, and the instance is set to auto-assign a public IP for internet accessibility. The security group is configured to allow SSH (port 22) access for remote login.



- **To modify the** IAM role **for your EC2 instance:**

  1. **Go to the** AWS IAM Console**, select** Roles**, and find the** flaskec2role**.**

  2. **Click** Attach Policies**, then choose the required policies (e.g.,** DynamoDBFullAccess**,** SNSFullAccess**) and click** Attach Policy**.**

  3. **If needed, update the instance to use this modified role by selecting the EC2 instance, clicking** Actions**, then** Security**, and** Modify IAM role **to select the updated role.**

- To connect to your EC2 instance:

  1. Go to the **EC2 Dashboard**, select your running instance, and click **Connect**.

  2. Follow the instructions provided in the **Connect To Your Instance** dialog, which will show the SSH command (e.g., ssh -i flaskkeypair.pem ec2-user@<public-ip>) to access your instance using the downloaded .pem key.

Now connect the EC2 with the files

EC2 > Instances > i-0caa76d6126ba3c8f > Connect to instance

Connect to instance info

Connect to your instance i-0caa76d6126ba3c8f (FlaskEC2Role) using any of these options

EC2 Instance Connect | Session Manager | SSH client | EC2 serial console

Instance ID
i-0caa76d6126ba3c8f (FlaskEC2Role)

Connection Type

- Connect using EC2 Instance Connect
  Connect using the EC2 Instance Connect browser-based client, with a public IPv4 or IPv6 address.
- Connect using EC2 Instance Connect Endpoint
  Connect using the EC2 Instance Connect browser-based client, with a private IPv4 address and a VPC endpoint.

Public IPv4 address
13.203.157.118

IPv6 address
–

Username
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.

ec2-user

Note: In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Cancel | Connect

CloudShell | Feedback | © 2025, Amazon Web Services, Inc. or its affiliates. | Privacy | Terms | Cookie preferences

https://us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-0b68d28a58547c0c8&osUser=ec2-us...

Account ID: 2210-8219-7973
rsoaccount-new/68034028d24717cbb9270727

```
       #_
   ~\_  ####          Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___    https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-31-101 ~]$
```

i-0b68d28a58547c0c8 (flaskec2role)
PublicIPs: 184.73.38.89 PrivateIPs: 172.31.31.101

CloudShell | Feedback | © 2026, Amazon Web Services, Inc. or its affiliates. | Privacy | Terms | Cookie preferences

Milestone 6: Deployment Using EC2

Activity 6.1: Install Software on the EC2 Instance

Install Python3, Flask, and Git

1. **On Amazon Linux 2:**

   sudo yum update -y

   sudo yum install python3 git

   sudo pip3 install flask boto3

2. **Verify Installations:**

   flask –version

   git --version

Activity 6.2: Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

- Run: 'git clone https://github.com/your-github-username/your-repository-name.git'
- Note: change your-github-username and your-repository-name with your credentials here: 'git clone https://github.com/Ravi-teja-777/medtrack_app.git
- This will download your project to the EC2 instance.

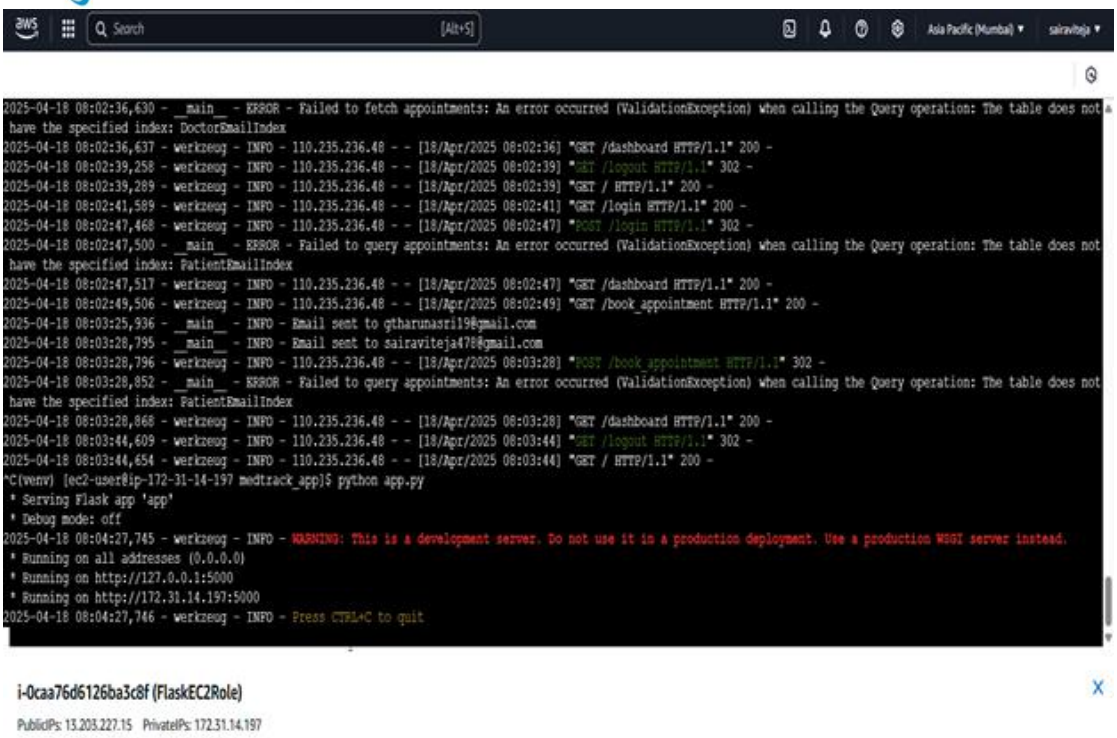
To navigate to the project directory, run the following command:

cd Medtrack

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

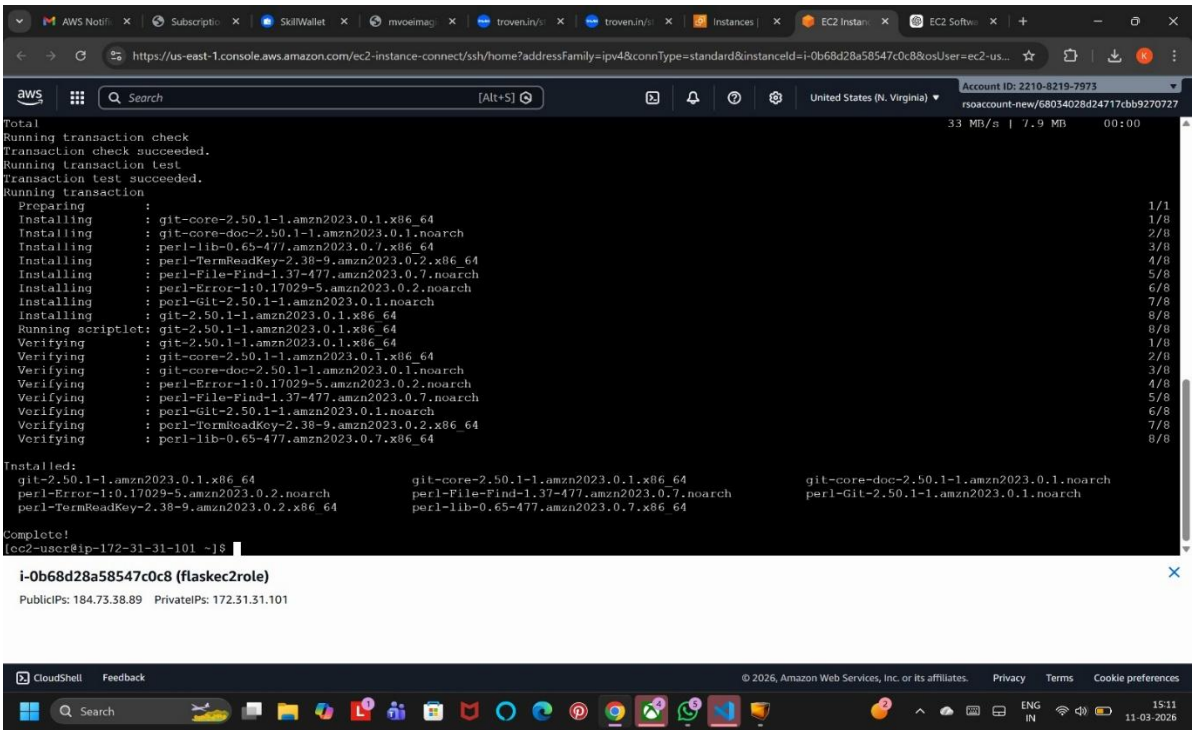
sudo flask run --host=0.0.0.0 --port=5000



Verify the Flask app is running:

http://your-ec2-public-ip

- Run the Flask app on the EC2 instance



Access the website through:

PublicIPs: https://13.201

Milestone 7: Testing and Deployment

Activity 7.1: Functional testing to verify the Project

Conduct functional testing to verify user registration, login, book requests, and notifications.

Home Page:

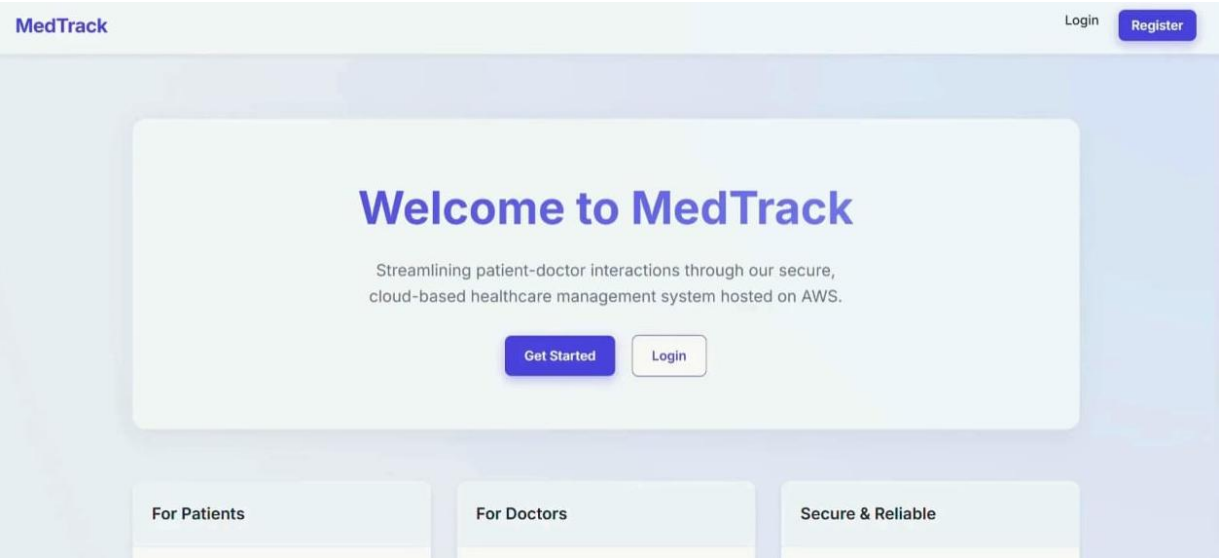
The Home Page of your project is the main entry point for users, where they can interact with the system. It typically includes:

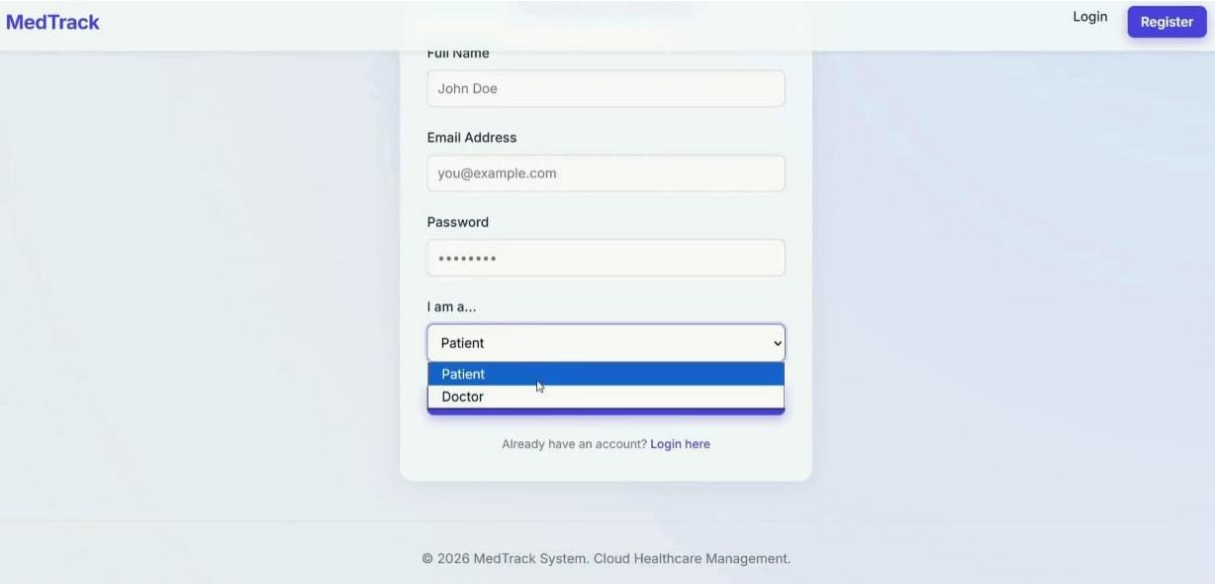
1. Input Fields: For users to enter basic information like appointment requests, diagnosis submissions, or service bookings.

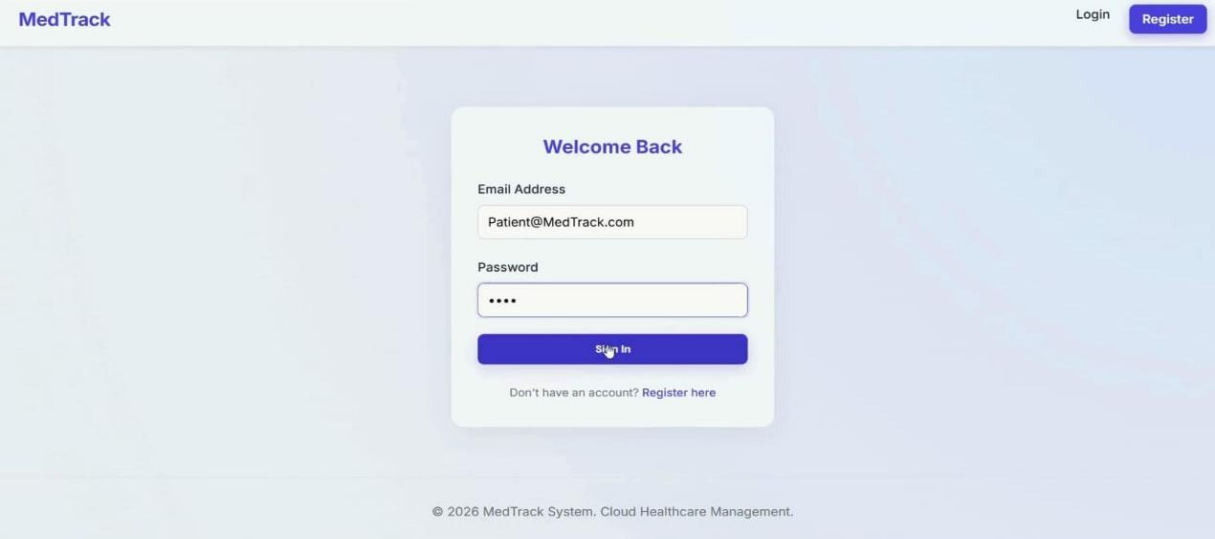
2. Navigation: Links to other sections such as the login page, dashboard, or service options.

3. Responsive Design: Ensures the page is accessible across devices with a clean, user-friendly interface.

The Home Page serves as the initial interface that directs users to the key functionalities of your web application.





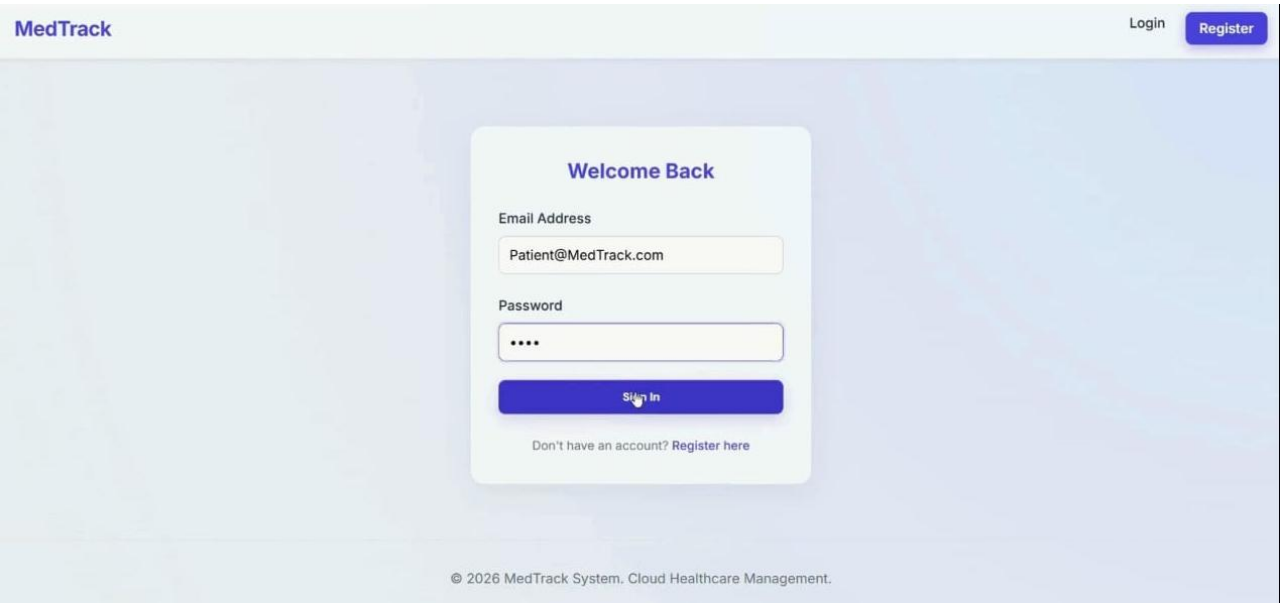


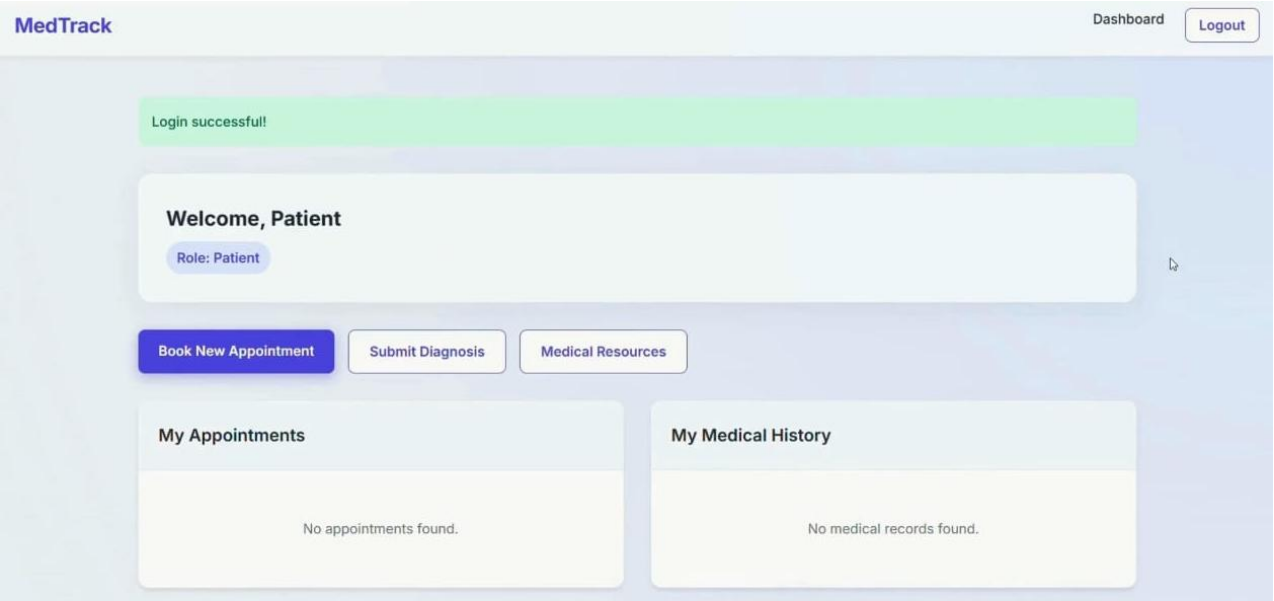
DOCTOR AND PATIENT REGISTRATION PAGE :

The Doctor Registration Page allows doctors to register and create an account on the platform. It typically includes:

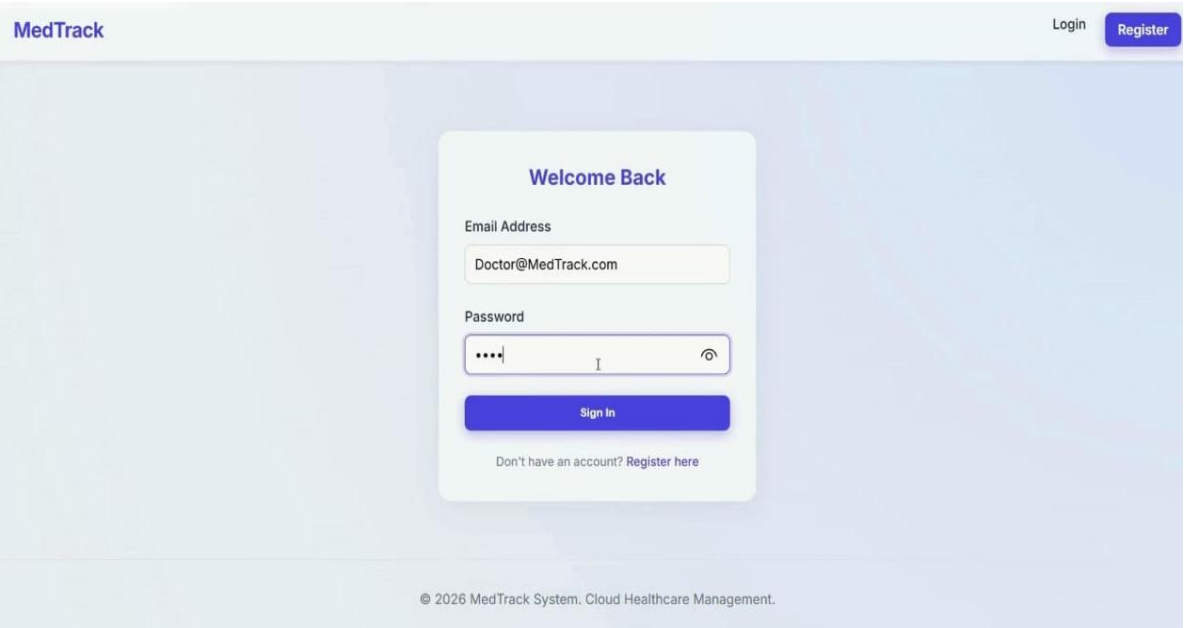
1. Input Fields: For doctor details such as name, specialty, qualifications, and contact information.
2. Login Credentials: Fields for setting a username and password for secure access.
3. Submit Button: A button to submit the registration details, which will then be stored in the database after validation.

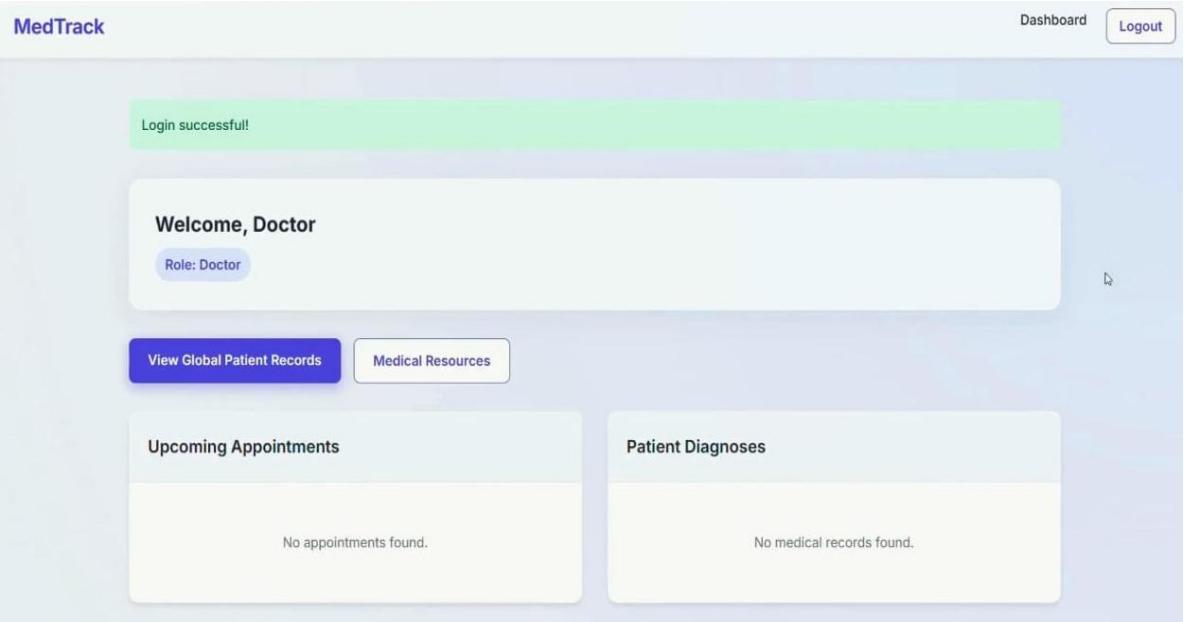
Patients log-in:





Doctors log-in:





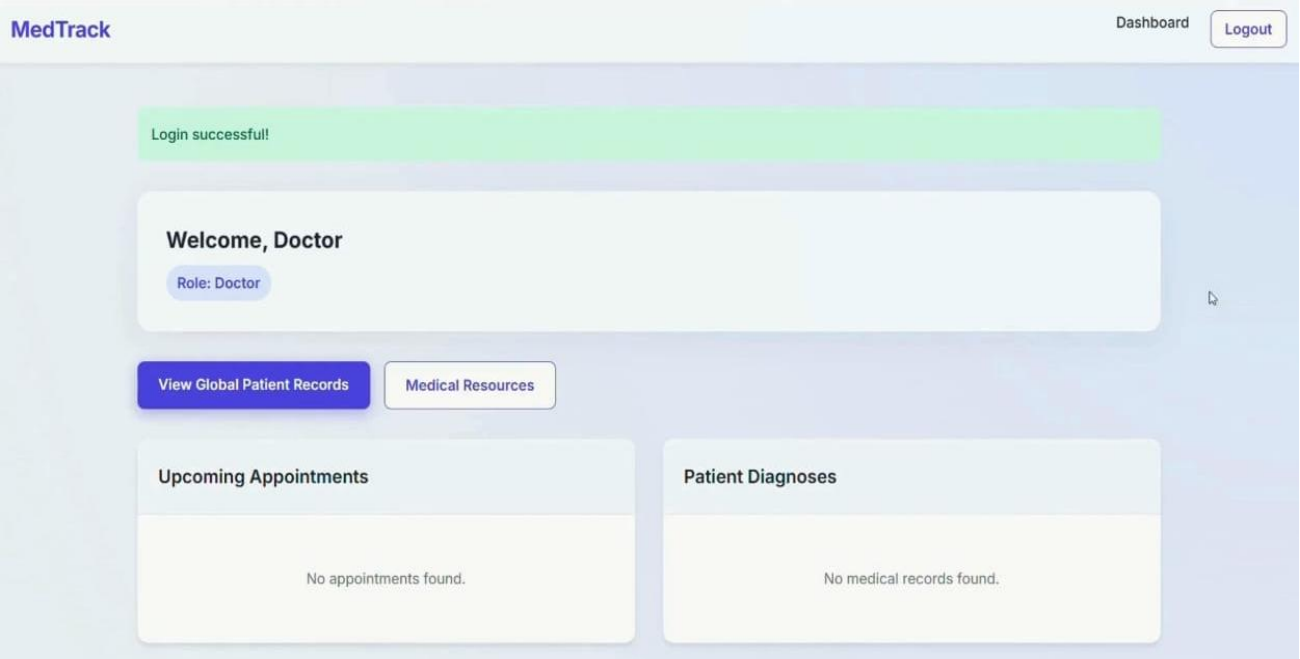
User Dashboard:

The User Dashboard (for patients) provides an easy interface to manage appointments and track their status. It typically includes:

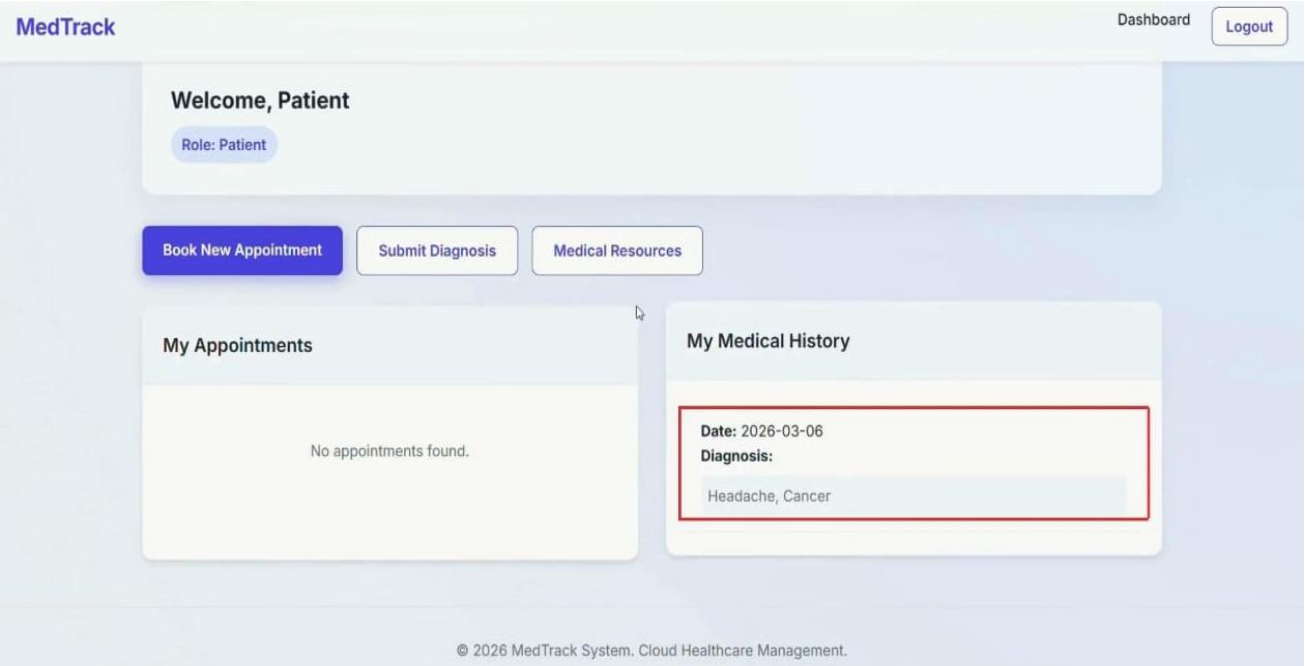
1. Book Appointment Section: A form for selecting a doctor, choosing an appointment time, and submitting the request.
2. Appointment Status: A section showing the current status of appointments (e.g., confirmed, pending, or completed) with options to view details or cancel.
3. Upcoming Appointments: A list of future appointments with relevant details such as doctor name, date, and time.

This dashboard helps patients book new appointments and keep track of their healthcare schedules.

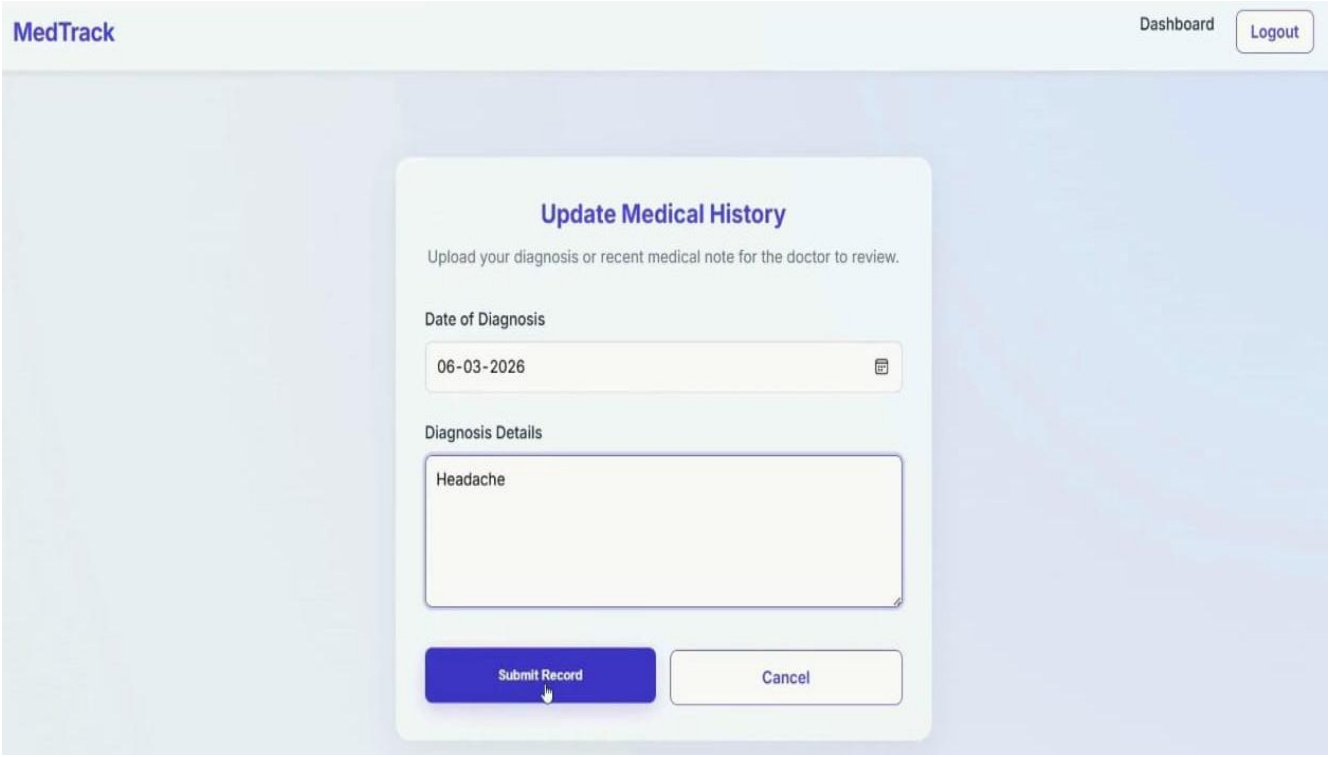
Doctor's Dashboard:



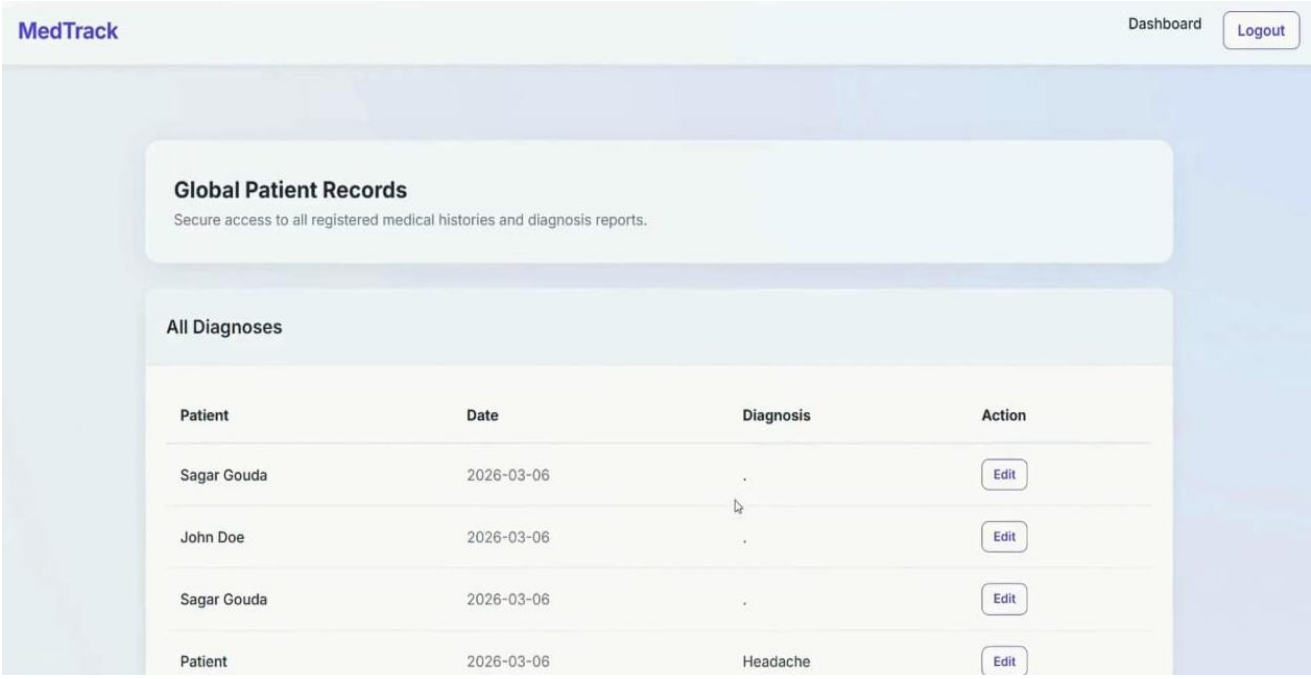
Patient's Dashboard:



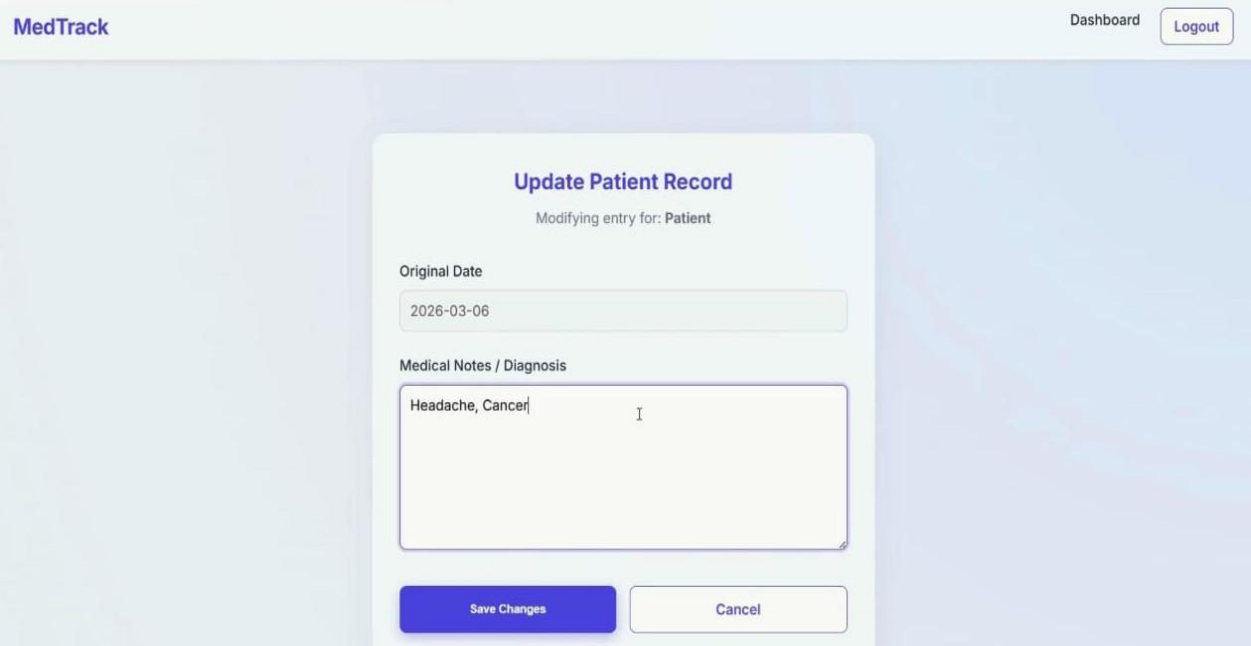
Update Medical History:



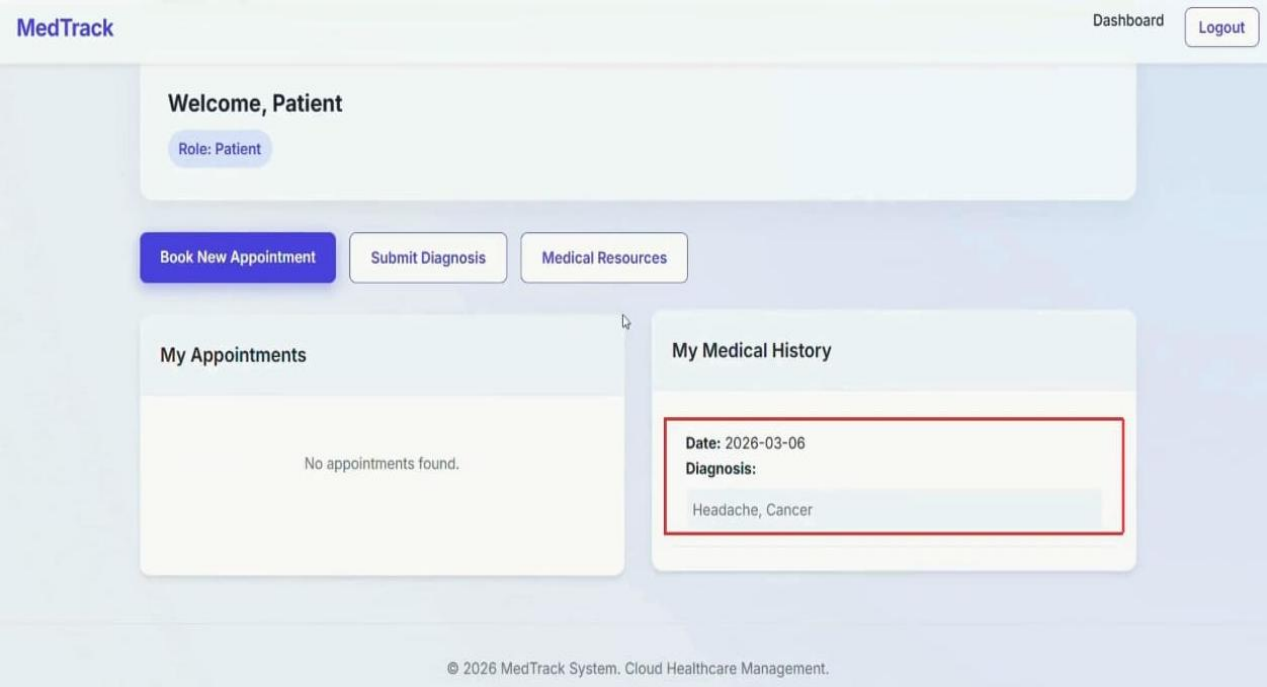
Patient's Record:



Update Patient's Record:



Patient's Medical History:



Dynamodb Database updations :

1. **Appointment table :**

Table: AppointmentsTable - Items returned (2)

Scan started on March 12, 2026, 19:27:09

Actions | Create item

1

appointment_id *(String)*	appointment_date	created_at	diagnosis	doctor_en
86d52cfe-cc66-46b0-941d-1…	2026-03-12	2026-03-12	–	gtharunars
42e13fef-fbe9-42d1-9905-f…	2026-03-12	2026-03-12	make sure t…	siri@thesn

Description: Store and manage patient appointment details, including appointment ID, appointment date, creation date, diagnosis information, and doctor email in the **AppointmentsTable** on DynamoDB for efficient scheduling and medical record tracking.

Conclusion:

The **MedTrack application** has been successfully developed and deployed using a robust cloud-based architecture tailored for modern healthcare environments. Leveraging AWS services such as EC2 for hosting, DynamoDB for secure and scalable patient data management, and SNS for real-time alerts, the platform ensures reliable and efficient access to essential medical tracking services. This system addresses critical challenges in healthcare such as managing patient records, monitoring medication schedules, and ensuring timely communication between healthcare providers and patients.

The cloud-native approach enables seamless scalability, allowing MedTrack to support increasing numbers of users and data without compromising performance or reliability. The integration of Flask with AWS ensures smooth backend operations, including patient registration, medication reminders, and health updates. Thorough testing has validated that all features—from user onboarding to alert notifications—function reliably and securely.

In conclusion, the MedTrack application delivers a smart, efficient solution for modernizing healthcare management, improving patient care, and streamlining communication between medical staff and patients. This project highlights the transformative power of cloud-based technologies in solving real-world challenges in the healthcare sector.